

Mining Statistically Likely k -Reachable States in Probabilistic Programs

Arnab Ray* Nitesh Trivedi† Ansuman Banerjee* Sourav Chakraborty*
Arijit Ghosh* Subhajit Roy†

Abstract

We propose the notion of *statistically likely k -step reachable set* in probabilistic programs, a statistically robust notion for high-probability k -step reachable program states. We design an inductive algorithm to capture this set as a symbolic representation in propositional logic for Boolean probabilistic programs. Our methodology iteratively learns a symbolic formula for *statistically likely k -step reachable set* that involves (a) learning an initial symbolic candidate via decision tree learning, (b) collecting positive and negative counterexamples via forward and backward verification checks and (c) refining the current candidate via a sequence of *prune* and *split* moves on the decision tree. We demonstrate that *statistically likely k -step reachable set* can reveal some interesting properties about programs by studying some probabilistic programs from literature.

1 Introduction

Symbolic analysis has significantly advanced the state-of-the-art for program verification. Such analyzers compute a symbolic representation of a target program, commonly in some fragment of logic, and use satisfiability engines (SAT solvers) to search for proofs and counterexamples. However, verification of real-world programs remains challenging. The primary difficulty is the challenge in modeling the uncertainties presented by the environment within which a program is executing. The environment can, though sparingly, provide unexpected values that deviate the program from its “regular” set of reachable states. For example, a memory allocation using a `malloc()` may infrequently fail (return NULL), that may require executing an exception handling routine.

A possible strategy is to model the environment as a stochastic process, where the probability distribution over different kinds of failures can be based on recommendation of domain experts or estimated from empirical studies. This allows us to ignore rare events and understand the *common-case* behaviors of the program. We refer to such a set of reachable states as *statistically likely k -step reachable set*.

In this work, we attempt to synthesize the *statistically likely k -step reachable set* for such probabilistic programs. Such reachable sets have applications in program understanding as the computed reachable sets do not include corner-case exception states. Further, along the lines of prior work like angelic verification [Das *et al.*, 2015; Das and Lal, 2017; Joshi *et al.*, 2012; Lahiri and Roy, 2022], the attempt to identify potential false positives that are signaled due to unduly demonic assumptions about the environment, *statistically likely k -step reachable set* can also have applications in sieving through false positives for real bugs. Our approach uses a combination of formal and learning based approaches to tackle this problem. Symbolically characterizing the common-case behaviors of probabilistic programs is a core problem in AI, with applications spanning Bayesian inference, AI safety, and bug prioritization. We address this by learning a *symbolic posterior summary* that distinguishes high-probability program behaviors from rare corner cases.

*Indian Statistical Institute Kolkata. {ansuman, sourav, arijitghosh}@isical.ac.in, rayarnab015@gmail.com

†Indian Institute of Technology Kanpur. {subhajit, nitesht}@iitk.ac.in

Contributions.

The contributions of this work are as follows:

- We propose the notion of *statistically likely k -step reachable set* $\epsilon\text{-Cl}_k(\varphi)$, a symbolic characterization of states that probabilistic programs reach with non-negligible probability;
- We prove that even the problem of deciding the emptiness of $\epsilon\text{-Cl}_k(\varphi)$ is itself computationally hard, which motivates approximate solutions;
- We design an algorithm to compute this set with statistical guarantees, which to our knowledge is the first framework to learn symbolic approximation of high-probability reachable sets;
- We build our algorithm into a tool, PRORP, and use it to study the *statistically likely k -step reachable set* on probabilistic programs from literature.

2 Preliminaries

2.1 Boolean Formulas, satisfiability and Samplers

For a Boolean formula φ , we use $\text{Supp}(\varphi)$ to denote the set of variables appearing in φ . An assignment $\sigma \in \{0, 1\}^{|\text{Supp}(\varphi)|}$ to $\text{Supp}(\varphi)$ is a *satisfying assignment* or *witness* if it makes φ evaluate to 1. We denote the set of all satisfying assignments of φ as R_φ and say a formula φ represents the set R_φ .

A formula is in **conjunctive normal form (CNF)** (resp. **disjunctive normal form (DNF)**) if it is a conjunction (resp. disjunction) of one or more clauses (resp. terms), where each clause (resp. term) is a disjunction (resp. conjunction) of literals. A **literal** is a variable or its negation.

Definition 1 (Uniform samplers). *A uniform-CNF-sampler (resp. uniform-DNF-sampler) is a randomized algorithm $\mathcal{A}$ that takes as input a CNF-formula (resp. DNF-formula) φ and outputs a satisfying assignment of φ uniformly at random from the set of all satisfying assignments. That is, for any $\sigma \in R_\varphi$, $\Pr[\mathcal{A} \text{ outputs } \sigma] = \frac{1}{|R_\varphi|}$, where the probability is over the internal randomness used in $\mathcal{A}$.*

For a CNF or DNF formula φ , by drawing m iid samples from $\mathcal{D}(R_\varphi)$ we mean using the relevant samplers to draw m independent and identically distributed samples from the distribution $\mathcal{D}$ over the set R_φ .

2.2 Probabilistic Programs and k -step reachability

Definition 2 (Probabilistic programs). *A probabilistic program is a tuple $(\mathcal{X}, \text{loc}, \text{trans})$ where (i) $\mathcal{X} = \{x_1, \dots, x_n\}$ is a finite set of Boolean program variables, (ii) loc is a finite set of control locations and (iii) trans is a finite set of transitions, each of the form $(\ell, \text{guard}) \Rightarrow \{(p_1, u_1, \ell'_1), \dots, (p_m, u_m, \ell'_m)\}$, where $\ell, \ell'_i \in \text{loc}$, $\sum_i p_i = 1$, each u_i is a deterministic update over $\mathcal{X}$, and when the Boolean predicate guard holds at ℓ , the program moves to ℓ'_i applying u_i with probability p_i .*

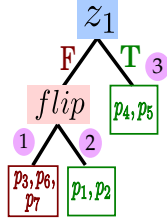
```
def Geo0(int z, Bool flip):
    while (flip == 0):
        d = bernoulli(0.5)
        if d:
            flip = 1
        else:
            z = z + 1
    return {z, flip}
```

Note that any program defined over variables taking values in a bounded integer domain can be encoded as a finite propositional formula via bit-blasting, a technique formalized in [Biere *et al.*, 1999]. This encoding also extends naturally to probabilistic programs over finite domains [Kwiatkowska *et al.*, 2011].

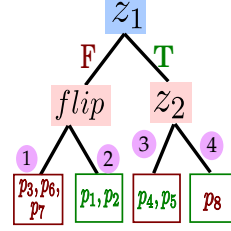
Figure 1: Program Geo0

Id	z2	z1	z0	flip	Label
p1	0	0	0	1	+
p2	0	0	1	1	+
p3	0	0	1	0	-
p4	0	1	0	1	+
p5	0	1	1	1	+
p6	1	0	0	0	-
p7	1	0	1	0	-

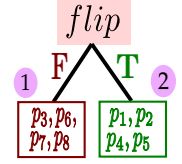
(a) Initial samples



(b) Initial Tree



(c) Split



(d) Prune

Figure 2: PRORP Repair Phase

Definition 3 (Program states). *Given a probabilistic program P defined over a set of Boolean program variables $\mathcal{X}$, a **program state** σ is a mapping from $\mathcal{X}$ to $\{0, 1\}^{|\mathcal{X}|}$, i.e., it is a valuation over the program variables. Thus, the entire program state space can be represented as $\Omega_P := \{0, 1\}^{|\mathcal{X}|}$.*

Example: The boolean probabilistic program Geo0 in Figure 1 is defined over a non-negative integer variable z and a Boolean flag $flip$. For a 3-bit encoding, $z = z_2 z_1 z_0$ with $\mathcal{X} = \{z_2, z_1, z_0, flip\}$, giving a state space $\Omega_{Geo0} = \{0, 1\}^4$. An example of a program state is $\{z_2 = 1, z_1 = 0, z_0 = 1, flip = 0\}$ which corresponds to $(z = 5, flip = 0)$.

Definition 4 (k -step transition and terminal states). *For a probabilistic program P defined over program variables $\mathcal{X}$ with Ω_P being the entire state space and for any $k \in \mathbb{N}$, the **k -step program transition probability function** is a function $\Gamma_P^k : \Omega_P \times \Omega_P \rightarrow [0, 1]$ such that $\Gamma_P^k(\sigma, \sigma')$ gives the probability of reaching state σ' from the state σ in exactly k steps. So clearly, for all $\sigma \in \Omega_P$, $\sum_{\sigma'} \Gamma_P^k(\sigma, \sigma') = 1$.*

Let P be a probabilistic program over program variables $\mathcal{X}$, and let Ω_P denote its state space. A state $\sigma \in \Omega_P$ is called a **terminal state** or sink state of P if no further program transitions are possible from σ , that is,

$$\forall \sigma' \in \Omega_P, \quad \Gamma_P^1(\sigma, \sigma') = \begin{cases} 1 & \text{if } \sigma' = \sigma, \\ 0 & \text{otherwise.} \end{cases}$$

We denote the set of all terminal states of P by $\mathcal{T}_P \subseteq \Omega_P$. Specifically, terminal states correspond to program configurations in which the program execution has halted.

Given $\sigma' \in \mathcal{T}_P$, $\Gamma_P^{\leq k}(\sigma, \sigma')$ denotes the probability of reaching σ' starting from σ in at most k steps, that is, $\Gamma_P^{\leq k}(\sigma, \sigma') := \Gamma_P^i(\sigma, \sigma')$, where $i = \min\{j \in [1, k] \mid \Gamma_P^j(\sigma, \sigma') > 0\}$, otherwise 0.

The **k -step program formula** F_P^k is a Boolean formula over $2|\mathcal{X}|$ number of variables representing a k -step single execution path of the program. In other words, for any $\sigma, \sigma' \in \Omega_P$, $F_P^k(\sigma, \sigma') = 1$ if $\Gamma_P^k(\sigma, \sigma') > 0$.

Definition 5 (Precondition). *For a probabilistic program P defined over program variables $\mathcal{X}$, with Ω_P being the entire state space, a **precondition set** is a subset R_φ of Ω_P describing a possible set of valuations at the beginning of P . The subset is described implicitly by the Boolean formula φ , known as the **precondition**.*

2.3 Compiling Decision Trees to Boolean Formula

A *decision tree* for a Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is a rooted binary tree whose internal nodes are labeled by variables $x_i \in \{x_1, \dots, x_n\}$, whose outgoing edges are labeled by the Boolean outcomes 0 and 1, and whose leaves are labeled by output values of f . Each root-to-leaf path encodes a conjunction of literals specifying an input $\mathbf{x} \in \{0, 1\}^n$, and the label of the leaf is the value $f(\mathbf{x})$. Equivalently, a decision

tree represents the function f as the disjunction of the conjunctions of the decisions along all root-to-leaf paths labeled 1 at the leaves.

For example, the tree in Figure 2b corresponds to the DNF $(z_1 \vee (\neg z_1 \wedge \text{flip}))$. For a given decision tree $\mathfrak{T}$, we assume that $\text{ExtractLeaves}(\mathfrak{T})$ returns the list of all leaves in $\mathfrak{T}$, and $\text{TreeDNF}(\mathfrak{T})$ outputs the DNF represented by $\mathfrak{T}$.

3 Statistically likely k -step reachable set

To formalize the notion of statistically likely k -step reachable set, we first define the concept of k -step reachability.

Definition 6 ((At most k)-step reachability). *For a precondition φ of a probabilistic program P , a k -step reachable formula with respect to precondition φ is a Boolean formula ζ that satisfies the following condition: $\sigma' \in R_\zeta \iff \exists \sigma \in R_\varphi$ with $\left(\bigvee_{i=0}^k F_P^i(\sigma, \sigma')\right) = 1$. Thus, R_ζ is the set of all terminal states that reachable in at most k steps if the program P is executed with a starting state in R_φ . We call this set the k -step reachable set and denote it by $\text{Cl}_k(\varphi)$.*

Definition 7 (Atmost k -step transition probability). *For a precondition φ consider the k -step reachable set $\text{Cl}_k(\varphi)$. Then for any $\sigma' \in \text{Cl}_k(\varphi)$ and any given probability distribution $\mathcal{D}$ over the set R_φ , we define the at most k -step program transition function w.r.t. to precondition φ as $\Gamma_{P,\varphi}(\sigma') := \mathbb{E}_{\sigma \sim \mathcal{D}(R_\varphi)} \left[\Gamma_P^{\leq k}(\sigma, \sigma') \right]$. This quantity gives the probability of reaching the terminal state σ' in at most k steps from an initial state $\sigma \in R_\varphi$ chosen according to $\mathcal{D}$.*

However, $\text{Cl}_k(\varphi)$ may contain many states with negligible reachability probability when P starts from R_φ . To capture the states reached with “non-negligible” probability, we introduce the concept of *statistically likely k -step reachable set*.

Definition 8 (statistically likely k -step reachable set). *For a probabilistic program P , a precondition φ and a threshold $\epsilon \in (0, 1]$, we consider the set $\epsilon\text{-Cl}_k(\varphi) \subseteq \text{Cl}_k(\varphi)$, which we call the ϵ -statistically likely k -step reachable set for P w.r.t. the precondition φ and is defined as $\epsilon\text{-Cl}_k(\varphi) := \text{Cl}_k(\varphi) \cap \{\sigma' \mid \Gamma_{P,\varphi}(\sigma') \geq \epsilon\}$.*

3.1 Exact synthesis of $\epsilon\text{-Cl}_k(\varphi)$

Given a probabilistic Boolean program P with a set of input variables $\mathcal{X}$, a distribution $\mathcal{D}$ from which $\mathcal{X}$ is sampled, precondition φ , $k \in \mathbb{N}$ representing the number of steps, and the parameter $\epsilon \in (0, 1]$, we refer to the task of *exactly synthesizing* the set $\epsilon\text{-Cl}_k(\varphi)$ as $\epsilon\text{-Cl}_k(\varphi)$ **SYNTHESIS**.

To establish the computational hardness of $\epsilon\text{-Cl}_k(\varphi)$ **SYNTHESIS**, we analyze the $\epsilon\text{-Cl}_k(\varphi)$ **EMPTINESS** problem, which asks whether $\epsilon\text{-Cl}_k(\varphi)$ is empty. Since the task of exactly synthesizing $\epsilon\text{-Cl}_k(\varphi)$ is at least as hard as solving $\epsilon\text{-Cl}_k(\varphi)$ **EMPTINESS**, hardness of the latter yields a lower bound for the former. We prove the PSPACE-hardness of $\epsilon\text{-Cl}_k(\varphi)$ **EMPTINESS** by a Karp reduction from the known PSPACE-complete problem **DFA-INTERSECTION** [Kozen, 1977]. We state the theorem below and provide the complete proof in Section 7.

Theorem 1. $\epsilon\text{-Cl}_k(\varphi)$ **EMPTINESS** is PSPACE-complete.

This result motivates approximate synthesis of $\epsilon\text{-Cl}_k(\varphi)$. We now formalize the problem of approximate synthesis.

3.2 Approximate synthesis of $\epsilon\text{-Cl}_k(\varphi)$

For a probabilistic program P and a precondition φ , a set $S \subseteq \text{Cl}_k(\varphi)$ is called a **ϵ -approximate k -step reachable set** if $\sum_{\sigma \in \text{Cl}_k(\varphi) \setminus S} \Gamma_{P,\varphi}(\sigma) \leq \epsilon$. A formula ψ representing an ϵ -approximate k -step reachable set R_ψ is called an **ϵ -approximate k -step reachable formula**. Note that, if S is any ϵ -approximate k -step reachable set, $\epsilon\text{-Cl}_k(\varphi) \subseteq S$. Trivially, the entire state space is an ϵ -approximate k -step reachable set. So our goal is to learn an ϵ -approximate k -step reachable formula ψ whose set R_ψ is close to $\epsilon\text{-Cl}_k(\varphi)$. To quantify closeness, we define two error measures between ψ and $\text{Cl}_k(\varphi)$.

Definition 9 (Error distances for candidate formula). *Given a probabilistic program P , a precondition φ and a distribution $\mathcal{D}$ over R_φ , the Fo-errors and Ba-errors between a candidate Boolean formula ψ and the k -step reachable set $\text{Cl}_k(\varphi)$ are quantified as follows*

- The Fo-error distance $\text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi))$ is the probability that the terminal state reached after execution of at most k steps of P on an initial state randomly picked from R_φ according to $\mathcal{D}$ is not satisfied by ψ . It is defined as $\text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi)) := \sum_{\sigma \in (\text{Cl}_k(\varphi) \setminus R_\psi)} \Gamma_{P,\varphi}(\sigma)$.
- The Ba-error distance $\text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi))$ is a measure of the fraction of program states in R_ψ that are not satisfied by $\text{Cl}_k(\varphi)$. It is defined as $\text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi)) := \Pr_{\sigma \sim_U R_\psi}[\sigma \notin \text{Cl}_k(\varphi)]$

Definition 10 ((ϵ_1, ϵ_2) -approximate statistically likely k -step reachability). *For any probabilistic program P , precondition φ , input distribution $\mathcal{D}$ over R_φ , $k \in \mathbb{N}$ and any $0 \leq \epsilon_1, \epsilon_2 \leq 1$, a formula ψ is called a **(ϵ_1, ϵ_2) -approximate statistically likely k -step reachable formula** of P with respect to φ if $\text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi)) < \epsilon_1$ and $\text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi)) < \epsilon_2$.*

Given P , precondition φ , $0 \leq \epsilon_1, \epsilon_2, \delta \leq 1$, and $k \in \mathbb{N}$ an algorithm is said to synthesize an **$(\epsilon_1, \epsilon_2, \delta)$ -approximate statistically likely k -step reachable set** of P with respect to φ if it outputs a Boolean formula ψ such that with probability at least $(1 - \delta)$, R_ψ is a (ϵ_1, ϵ_2) -approximate statistically likely k -step reachable formula of P with respect to φ .

Let us define the problem of approximating $\epsilon\text{-Cl}_k(\varphi)$.

Problem statement: Given a probabilistic Boolean program P with input variables $\mathcal{X}$, precondition φ , input distribution $\mathcal{D}$ over R_φ , $k \in \mathbb{N}$ representing the number of steps, and error-bounds $\epsilon_1, \epsilon_2, \delta \in (0, 1]$, we want to synthesize an $(\epsilon_1, \epsilon_2, \delta)$ -approximate statistically likely k -step reachable set of the probabilistic program P with respect to φ .

Clearly, for the given problem setup, the set $\epsilon_1\text{-Cl}_k(\varphi) \subseteq S$, where S is some $(\epsilon_1, \epsilon_2, \delta)$ -approximate statistically likely k -step reachable set of P with respect to φ . To solve the above problem, we introduce an algorithm PRORP. We now study the output of PRORP using an illustrative example.

Illustrative Example. We illustrate the notion of approximate statistically likely k -step reachable set using the probabilistic program Geo0 (Figure 1). Starting from $z = 0$, the program returns $z = t$ with probability 2^{-t} since termination requires t tails before a head, so small values occur frequently while larger ones are exponentially unlikely. This creates a structural asymmetry: high-order bits rarely change whereas low-order bits vary often. For a precondition φ , let R_φ denote the initial states, and let $\epsilon\text{-Cl}_k(\varphi)$ be the states reachable within k iterations with probability at least ϵ . For Geo0, while $\text{Cl}_k(\varphi)$ contains many low-probability states, $\epsilon\text{-Cl}_k(\varphi)$ remains small and captures the common-case behavior.

The results (z being a 16-bit unsigned integer) in Table 1 show that PRORP closely tracks $\epsilon\text{-Cl}_k(\varphi)$. For preconditions involving higher-order bits (z_{14}, z_{12}, z_{10}), the learned candidate exactly characterizes the likely reachable states. As the precondition moves to lower-order bits, chances of leaving the initial region

Table 1: Output of PRORP on Geo0 ($\epsilon_1 = \epsilon_2 = \epsilon = 0.05$)

φ	R_φ	ψ
z_{14}	$\bigcup_{j=0}^1 [(2j+1)2^{14}, (j+1)2^{15} - 1]$	z_{14}
z_{12}	$\bigcup_{j=0}^7 [(2j+1)2^{12}, (j+1)2^{13} - 1]$	z_{12}
z_{10}	$\bigcup_{j=0}^{31} [(2j+1)2^{10}, (j+1)2^{11} - 1]$	z_{10}
z_8	$\bigcup_{j=0}^{127} [(2j+1)2^8, (j+1)2^9 - 1]$	$z_8 \vee (\bar{z}_7 \wedge \bar{z}_6 \wedge \bar{z}_0)$
z_6	$\bigcup_{j=0}^{511} [(2j+1)2^6, (j+1)2^7 - 1]$	True

increases, so the formulas admit more states. In the extreme case z_6 , almost the entire state space becomes likely reachable, and PRORP returns True. These observations highlight the common-case behaviour of Geo0.

4 Algorithm

We use an iterative refinement strategy which “guesses” a potential candidate, and then, our verification subroutines validate the solution. Any counterexample augment the dataset and the process is repeated till a candidate is verified.

4.1 The verification subroutines

A candidate formula ψ for the $(\epsilon_1, \epsilon_2, \delta)$ -statistically likely k -step reachable set problem can have errors in two directions:

- The formula ψ excludes program states reachable from the inputs; we call such examples Fo-counterexamples.
- The formula ψ includes program states not reachable from the inputs; such examples are Ba-counterexamples.

We design two verification subroutines, FoRT (FoRward-Reachability-Tester) to test for Fo-errors and BaRT (BaCKward-Reachability-Tester) to test for Ba-errors. Our algorithm alternates between these two subroutines to obtain the two types of counterexamples.

4.1.1 FoRT

This subroutine (Algorithm 1) takes as input a probabilistic program P , a precondition φ (in CNF), an input distribution $\mathcal{D}$ over R_φ , a candidate formula ψ (in DNF), a closeness parameter ϵ , a confidence parameter δ and the number of steps k . It returns an estimate Error_1 of the Fo-error and a multiset CE_{Fo} of Fo-counterexamples. To capture “high-probability” counterexamples, FoRT samples a set U of initial states from $\mathcal{D}(R_\varphi)$ (Line 2), executes P for at most k steps (represented by $P^{\leq k}(u)$ for $u \in U$ in Line 5), and records the reached terminal states $y_1, \dots, y_m$ (possibly with repetitions). Each violating y_i is added to CE_{Fo} as a Fo-counterexample (Line 7); Error_1 is the fraction of samples failing ψ (Line 8).

Sample size m is chosen so that, with probability at least $(1 - \delta)$, Error_1 is a ϵ -additive approximation of the true Fo-error. Please note that FoRT only requires a black-box sampling access to $\mathcal{D}$.

Algorithm 1 FoRT($P, \varphi, \mathcal{D}, \psi, \epsilon, \delta, k$)

```
1:  $m \leftarrow \lceil \frac{2}{\epsilon^2} \log \frac{4}{\delta} \rceil$ 
2:  $U \leftarrow m$  iid samples from  $\mathcal{D}(R_\varphi)$ 
3:  $\text{CE}_{\text{Fo}} \leftarrow \emptyset, \text{Error}_1 \leftarrow 0$ 
4: for  $u \in U$  do
5:    $y \leftarrow P^{\leq k}(u)$ 
6:   if  $y \notin R_\psi$  then
7:      $\text{CE}_{\text{Fo}} \leftarrow \text{CE}_{\text{Fo}} \cup \{y\}$ 
8:      $\text{Error}_1 \leftarrow \text{Error}_1 + \frac{1}{m}$ 
9: return ( $\text{Error}_1, \text{CE}_{\text{Fo}}$ )
```

Algorithm 2 BaRT($P, \varphi, \psi, \epsilon, \delta, k$)

```
1:  $m \leftarrow \lceil \frac{2}{\epsilon^2} \log \frac{4}{\delta} \rceil$ 
2:  $V \leftarrow m$  iid samples from  $\mathcal{U}(R_\psi)$ 
3:  $\text{CE}_{\text{Ba}} \leftarrow \emptyset, \text{Error}_2 \leftarrow 0$ 
4: for  $v \in V$  do
5:    $\Upsilon \leftarrow \varphi(X_0) \wedge \bigwedge_{j < k} F_P(X_j, X_{j+1}) \wedge \bigvee_{j \leq k} (X_j = v)$ 
6:   if  $\Upsilon$  is UNSAT then
7:      $\text{CE}_{\text{Ba}} \leftarrow \text{CE}_{\text{Ba}} \cup \{v\}$ 
8:      $\text{Error}_2 \leftarrow \text{Error}_2 + \frac{1}{m}$ 
9: return ( $\text{Error}_2, \text{CE}_{\text{Ba}}$ )
```

4.1.2 BaRT

This subroutine (Algorithm 2) takes as input a probabilistic program P , a precondition φ (in CNF), a candidate formula ψ (in DNF), a closeness parameter ϵ , a confidence parameter δ and the number of steps k . It returns an estimate Error_2 of the Ba-error and a multiset CE_{Ba} of Ba-counterexamples. Recall that Ba-counterexamples are states in $(R_\psi \setminus \text{Cl}_k(\varphi))$, and true Ba-error is $\frac{|R_\psi \setminus \text{Cl}_k(\varphi)|}{|R_\psi|}$. To generate Ba-counterexamples, BaRT samples m states from R_ψ using a uniform-like DNF-sampler (Line 2) and checks if the samples are in $\text{Cl}_k(\varphi)$ via a SAT-solver (Line 6). The formula Υ_i (in line 5) encodes “is the sampled state v in $\text{Cl}_k(\varphi)$.” If v is not in $\text{Cl}_k(\varphi)$ then it is a Ba-counterexample and is added to the multi-set CE_{Ba} (in line 7) and the Error_2 (updated in line 8) records what fraction of the samples that are drawn from R_ψ are not in $\text{Cl}_k(\varphi)$. Since our objective is to track the unreachable states in R_ψ , we give equal importance to these states by sampling from the uniform distribution $\mathcal{U}$ over R_ψ .

We choose sample size m so that with probability at least $(1 - \delta)$, Error_2 is a ϵ -additive approximation of true Ba-error.

4.2 The refinement subroutine

TreeRepair (Algorithm 3) repairs the decision-tree representation of a candidate formula ψ using Fo and Ba counterexamples. It takes as input a decision tree $\mathcal{T}$, counterexample sets CE_{Fo} and CE_{Ba} , thresholds Th_1, Th_2 , and the variable set $\mathcal{X}$. It aims to modify $\mathcal{T}$ so that no more than Th_1 members of CE_{Fo} and Th_2 members of CE_{Ba} remain misclassified.

The algorithm operates in rounds. At each round it first identifies the set L of current leaves using `ExtractLeaves` (line 4) and considers every potential refinement $(\ell, \mu) \in L \times (\{\text{Prune}\} \cup (\{\text{Split}\} \times \mathcal{X}))$. Each refinement is one of:

- **Prune**: delete a leaf node ℓ and then reclassifying all states assigned to that subtree. E.g., pruning node id 3 reduces both Error_1 and Error_2 to 0 (Figure 2d). The cost of the prune operation is 1.
- **Split**: replace a leaf ℓ with an internal decision on some variable in $\mathcal{X}$, creating two new leaves. For example, splitting node id 3 on predicate flip also eliminates all errors (Figure 2c). Splitting incurs cost 2.

For each potential refinement (ℓ, μ) , `GetGain` (line 6) computes the number of counterexamples that would be correctly classified if the refinement were applied, and `GetCost` (line 7) returns the “cost” for making the refinement corresponding to (ℓ, μ) . Finally, in line 8, a ratio $\text{Impact}_{(\ell, \mu)}$ of the gain to cost is calculated for each potential refinement. A refinement is sampled randomly with probability proportional to Impact (line 9) and applied via `MakeTheMove` (line 10). The set of counterexamples is appropriately refined

Algorithm 3 TreeRepair($\mathcal{T}_{\text{init}}, \text{CE}_{\text{Fo}}, \text{CE}_{\text{Ba}}, \text{Th}_1, \text{Th}_2, \mathcal{X}$)

```
1: Initialize  $\mathcal{T} \leftarrow \mathcal{T}_{\text{init}}$ 
2: Initialize  $E_1 \leftarrow \text{CE}_{\text{Fo}}, E_2 \leftarrow \text{CE}_{\text{Ba}}$ 
3: while NOT ( $|E_1| \leq \text{Th}_1$  and  $|E_2| \leq \text{Th}_2$ ) do
4:    $L \leftarrow \text{ExtractLeaves}(\mathcal{T})$ 
5:   for  $(\ell, \mu) \in L \times (\{\text{Prune}\} \cup (\{\text{Split}\} \times \mathcal{X}))$  do
6:      $\text{Gain}_{(\ell, \mu)} \leftarrow \text{GetGain}(\mathcal{T}, E_1, E_2, \ell, \mu)$ 
7:      $\text{Cost}_{(\ell, \mu)} \leftarrow \text{GetCost}(\mathcal{T}, E_1, E_2, \ell, \mu)$ 
8:      $\text{Impact}_{(\ell, \mu)} \leftarrow \frac{\text{Gain}_{(\ell, \mu)}}{\text{Cost}_{(\ell, \mu)}}$ 
9:   Sample  $(\ell, \mu)$  with probability proportional to  $\text{Impact}_{(\ell, \mu)}$ 
10:   $\mathcal{T}, E_1, E_2 \leftarrow \text{MakeTheMove}(\mathcal{T}, E_1, E_2, \ell, \mu)$ 
11: Output  $\mathcal{T}$ 
```

by removing those that have already been addressed. Refinements occur in a top–down manner: shallow splits result in coarse approximations of the reachable region, whereas deeper splits produce fine-grained corrections.

These rounds continue till remaining counterexamples in CE_{Fo} and CE_{Ba} fall below their thresholds.

4.3 The Learning Algorithm

Our main learning algorithm, PRORP, takes as input a probabilistic program P (over a set of variables $\mathcal{X}$), a precondition given as a CNF-formula φ , the input distribution $\mathcal{D}$ over R_φ , two error parameters (a Fo-error parameter ϵ_1 and a Ba-error parameter ϵ_2), a confidence parameter δ , and the number of steps k . The subroutine returns a DNF-formula that with probability at least $(1 - \delta)$ represents a (ϵ_1, ϵ_2) -approximate *statistically likely k -step reachable set* of P with respect to φ . PRORP does so by learning the corresponding decision tree.

PRORP starts by obtaining states from $\text{Cl}_k(\varphi)$ (positive examples; in line 4) and states from outside of $\text{Cl}_k(\varphi)$ (negative examples; in line 5) by calling the subroutines FoRT with the candidate formula set as `False` and BaRT with the candidate formula set as `True` respectively. For the initial sampling phase, the value of ϵ is chosen heuristically as $\frac{0.1}{\sqrt{\text{var}(|\mathcal{X}|)}}$ to account for programs with large state space as well as to collect “enough” labeled examples. Using the subroutine `CreateTree` a decision tree is learned that is consistent with both the positive and negative examples.

Once the initial decision tree is obtained, PRORP computes the corresponding DNF-formula using the subroutine `TreeDNF` (in line 8). Then it uses subroutines FoRT and BaRT (in lines 10–11) to estimate the Fo/Ba errors and construct the Fo/Ba counterexamples. If the errors are not within the acceptable limits the set of counterexamples are given to `TreeRepair` (in line 9) that appropriately modifies the tree and returns a new tree. This process is repeated until the errors fall within acceptable bounds and in practice for our experiments, we set a `timeout`. Finally, the DNF corresponding to the tree is returned.

Termination and Convergence. When only splits are allowed, termination follows from monotone growth of the tree over the finite state space $\{0, 1\}^{|\text{var}|}$. With both split and prune operations, memoization of visited trees can prevent cycling. Our refinement procedure thus deliberately adopts a global guess-and-check strategy rather than greedy ascent, since the non-convex search space admits local minima.

4.4 Illustrative Example

We illustrate PRORP on Geo0 (Figure 1) under precondition $\varphi : (z = 0) \wedge (\text{flip} = 0)$ using unsigned 3-bit encoding for z to analyze common-case reachability within 5 iterations under error thresholds $\epsilon_1 = \epsilon_2 = 0.2$.

Algorithm 4 PRORP($P(\mathcal{X}), \varphi, \mathcal{D}, \epsilon_1, \epsilon_2, \delta, k$)

```
1: Error1  $\leftarrow$  1, Error2  $\leftarrow$  1
2: CEFo  $\leftarrow$   $\emptyset$ , CEBa  $\leftarrow$   $\emptyset$ 
3: Th1  $\leftarrow$   $\lceil \frac{2}{\epsilon_1} \log(\frac{4}{\delta}) \rceil$ , Th2  $\leftarrow$   $\lceil \frac{2}{\epsilon_2} \log(\frac{4}{\delta}) \rceil$ 
4: PosExamples  $\leftarrow$  FoRT( $P, \varphi, \mathcal{D}, \text{False}, \frac{0.1}{\sqrt{|\mathcal{X}|}}, 0.1, k$ )
5: NegExamples  $\leftarrow$  BaRT( $P, \varphi, \text{True}, \frac{0.1}{\sqrt{|\mathcal{X}|}}, 0.1, k$ )
6:  $\mathfrak{T} \leftarrow$  CreateTree(PosExamples, NegExamples)
7: while NOT (Error1  $\leq$   $\epsilon_1$  and Error2  $\leq$   $\epsilon_2$ ) do
8:    $\psi \leftarrow$  TreeDNF( $\mathfrak{T}$ )
9:    $\mathfrak{T} \leftarrow$  TreeRepair( $\mathfrak{T}, \text{CE}_{\text{Fo}}, \text{CE}_{\text{Ba}}, \text{Th}_1, \text{Th}_2, \mathcal{X}$ )
10:   Error1, CEFo  $\leftarrow$  FoRT( $P, \varphi, \mathcal{D}, \psi, \epsilon_1, \delta, k$ )
11:   Error2, CEBa  $\leftarrow$  BaRT( $P, \varphi, \psi, \epsilon_2, \delta, k$ )
12: Output TreeDNF( $\mathfrak{T}$ )
```

To generate a labeled dataset of positive (reachable) and negative (unreachable) examples, we call FoRT with the candidate formula set to `False` and BaRT with the candidate set to `True`. The dataset is shown in Figure 2a, from which we learn a decision tree (Figure 2b). The green (positive) and red (negative) states are compiled into the formula $\psi : (z_1 \vee \text{flip})$ as described in Section 2.3.

The verification–repair loop continues until both checks succeed. In this instance, FoRT produces no counterexamples, while BaRT reports 1100 as a Ba-counterexample, raising Error₂ from 0.0 to 0.4. TreeRepair considers two repairs of leaf 3: (i) splitting on `flip` (Figure 2c) or (ii) pruning (Figure 2d), both driving (Error₁, Error₂) to 0. Assuming the prune mutation is chosen, the resulting tree (Figure 2d) represents `flip`. Re-verification yields a new Ba-counterexample 1101, prompting a refinement (e.g., splitting leaf 2 on z_2) that produces the final tree for $\psi := (\neg z_2 \wedge \text{flip})$. The verification errors now lie within the thresholds $(\epsilon_1, \epsilon_2) = (0.2, 0.2)$, and the algorithm terminates.

4.5 Theoretical Guarantee

Theorem 2 (Correctness of PRORP). *If PRORP halts, the returned formula ψ represents a $(\epsilon_1, \epsilon_2, \delta)$ -approximate statistically likely k -step reachable set.*

Corollary 1. *If PRORP halts and returns the formula ψ , then with probability at least $(1 - \delta)$, $\epsilon_1\text{-CL}_k(\varphi) \subseteq R_\psi$.*

The formal proof of Theorem 2 is given in Section 8.

5 Evaluation

PRORP¹ is implemented in Python using scikit-learn for decision trees, CMSGen [Golia *et al.*, 2021] and Pepin [Soos *et al.*, 2023] for uniform-like CNF/DNF sampling, CadiCaL [Biere *et al.*, 2024] for SAT solving, and Z3 [Moura and Bjørner, 2008] to generate program formula. We evaluate 16 benchmarks from prior work [Bao *et al.*, 2024; Chen *et al.*, 2015; Kaminski and Katoen, 2017] by varying the precondition φ , unroll depth $k \in \{16, 32, 64, 128\}$, bit-width (16/32-bit), using $\epsilon_1 = \epsilon_2 = 0.05$, $\delta = 0.1$ and uniform input distribution. We set a timeout of 2 hours.

Our experiments address four research questions:

- **RQ1:** Can PRORP synthesize an $(\epsilon_1, \epsilon_2, \delta)$ -approximate statistically likely k -step reachable set for a range of programs and preconditions?

¹Source code available on https://github.com/arnab-007/PRORP_.git

Table 2: Results on programs modelled with 16-bit integers

Program	Precondition	k	#PH	TT (secs)	Candidate
Geo0	(z_{14})	64	1	4.52	(z_{14})
LinExp	$(\text{count} < 64)$	64	3	44.33	$0 \leq \text{count} < 256$
Sum0	$(x = 64, n < \frac{k}{2})$	64	2	20.41	$0 \leq x < 2048$
Fair	$(\text{count} < 64)$	64	1	7.71	$0 \leq \text{count} < 128$
DepRV	$(x = 0, y = 0, n = k)$	64	6	75.40	$18 \leq x < 40, y = 64 - x$
RevBin	$(x = \frac{k}{2}, z < 16)$	64	3	35.23	$48 \leq z < 80$
Mart	$(c = 0, b = 64, r = 0)$	64	6	52.53	$c = 64, 0 \leq r < 24$

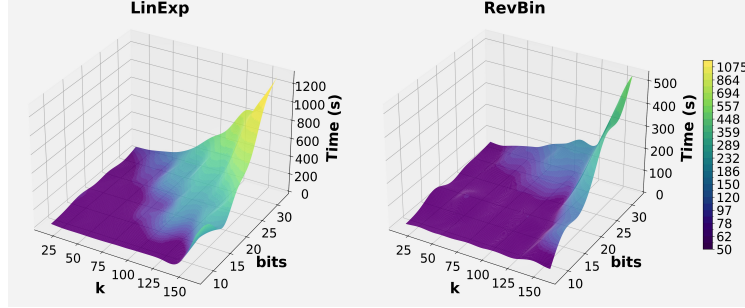


Figure 3: Scalability plots for LinExp and RevBin with preconditions $(\text{count} = 0, n < k)$ and $(x < \frac{k}{2}, z = 0)$ respectively

- **RQ2:** Does PRORP scale for many variables and large k ?
- **RQ3:** Do candidate sets synthesized by PRORP correspond to meaningful interpretations?
- **RQ4:** Is PRORP better than naïve Monte Carlo sampling?

5.1 RQ1 : Efficiency of synthesizing approximate statistically likely k -step reachable set

PRORP successfully produced a candidate formula for the $(\epsilon_1, \epsilon_2, \delta)$ -approximate k -reachable set on all 218 instances, with runtimes ranging from 4 to 755 seconds for the 16-bit encodings and 4 to 1885 seconds for the 32-bit encodings. We manually verified that, for each problem instance, the candidate set indeed contains the entire $\epsilon_1\text{-C1}_k(\varphi)$ set. Table 2 reports the number of learning phases (#PH), total time (TT) and candidate set obtained for seven illustrative benchmarks with a chosen precondition for 16-bit encoding and a given unrolling depth k . The complete and detailed set of experimental results is given in Section 10.

5.2 RQ2 : Scalability of PRORP with respect to k and bit-width of program variables

To address this question, we selected two representative benchmarks, LinExp and RevBin, each with a single precondition under a uniform input distribution, and measured runtime across varying bit-widths and unrolling depths. Figure 3 reports the resulting performance curves. As expected, runtime increases with both program bit-width and unrolling depth k : larger bit-width blows up the state space exponentially, while higher value of k expands the reachable portion of the state space. From the evaluation results, LinExp incurs the highest runtime cost, exceeding 1000 seconds for large k and bit-width, whereas RevBin exhibits a similar upward trend in runtime but remains under 500 seconds. Overall, these results indicate that PRORP scales predictably across varying bit-widths and unrolling depths for the evaluated programs, within the parameter ranges studied.

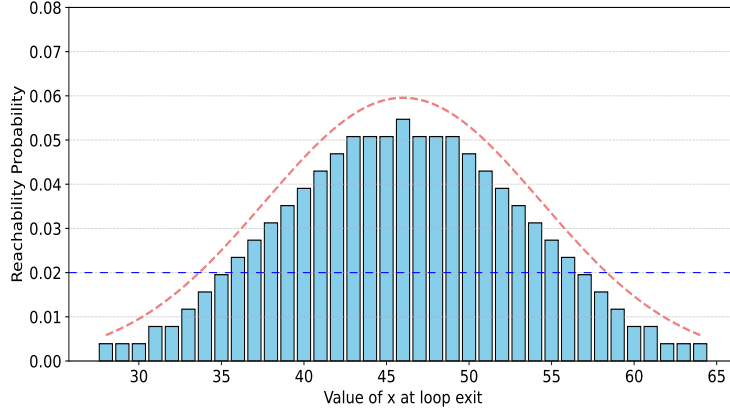


Figure 4: Sum0 for $R_\varphi := (x = 28, n = 8)$

```
def Sum0(int x, int n):
    while (n > 0):
        d = bernoulli(0.5)
        if (d):
            x = x + n
            n = n - 1
    return {x, n}
```

Figure 5: Program Sum0

```
def Unif2(int x, int l):
    y = 0; n = 32-1;
    while (n > 0):
        d = bernoulli(0.5)
        if d:
            y = 2y
        else:
            y = 2y + 1
            n = n - 1
    x = x + y
    return x
```

Figure 6: Program Unif2

5.3 RQ3 : Interpretation of the candidate sets to interesting problems (Case study)

Let us discuss and interpret the output of PRORP on the program Sum0 [Chen *et al.*, 2015]. The program (Figure 5) has a loop whose number of iterations is fixed by the input variable n . In the i -th iteration the value of x is incremented by $(n - i)$ with probability $\frac{1}{2}$ and not incremented with remaining probability $\frac{1}{2}$. Here, we consider only preconditions with exactly one satisfying assignment, i.e., $(x = x_0, n = n_0)$.

Note that for Sum0, all terminal states are characterized by $n = 0$. Let X be the random variable denoting the value of x for terminal states. Thus, $\mathbb{E}[X] = x_0 + \frac{n_0(n_0+1)}{4}$ and X takes values in the interval $[x_0, x_0 + \frac{n_0(n_0+1)}{2}]$. That is, if $R_\varphi = \{(x_0, n_0)\}$, then $\text{Cl}_k(\varphi) := x_0 \leq x \leq x_0 + \frac{n_0(n_0+1)}{2}$, assuming $k \geq n_0$. We also note that the random variable X follows a symmetric distribution, with $\Pr[X = x_0 + t] = \Pr[X = x_0 + \frac{n_0(n_0+1)}{2} - t]$. In Figure 4 the distribution is plotted for $(x_0, n_0) = (28, 8)$ which is clearly a bell-shaped curve. Thus, for any $\epsilon_1 \in (0, 1]$, $\epsilon_1\text{-Cl}_k(\varphi)$ is of the form $(x_0 + \frac{n_0(n_0+1)}{4} - t \leq x \leq x_0 + \frac{n_0(n_0+1)}{4} + t)$ which is obtained by chopping off the tails of the distribution.

We observe that the set represented by the output formula is close to $\epsilon_1\text{-Cl}_k(\varphi)$. We run PRORP on Sum0 for 4 preconditions with $k = 16$, each with a single satisfying assignment and parameters $\epsilon_1 = \epsilon_2 = 0.02$ and $\delta = 0.1$. The inputs x and n are 8-bit unsigned integers. The results of our experiment are compiled in Table 3. For each of the 4 preconditions, the right-hand tail of $\text{Cl}_k(\varphi)$ gets chopped off in R_ψ . Also, for two preconditions: $\{(x = 28, n = 8) \text{ and } (x = 31, n = 16)\}$, both the tails of $\text{Cl}_k(\varphi)$ are trimmed in R_ψ . However, for $R_\varphi : (x = 15, n = 16)$, R_ψ includes a few unreachable states and elongates the left-hand tail of $\text{Cl}_k(\varphi)$.

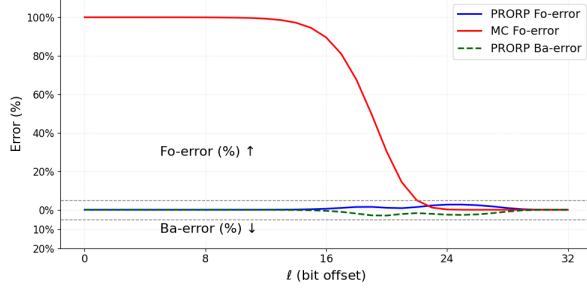


Figure 7: Fo and Ba error % for PRORP versus Monte Carlo

R_φ	$\text{Cl}_k(\varphi)$	$\epsilon_1\text{-Cl}_k(\varphi)$	R_ψ
($x = 28, n = 8$)	$28 \leq x \leq 64$	$36 \leq x \leq 56$	$32 \leq x \leq 63$
($x = 48, n = 16$)	$48 \leq x \leq 184$	$96 \leq x \leq 136$	$48 \leq x \leq 167$
($x = 31, n = 16$)	$31 \leq x \leq 167$	$79 \leq x \leq 119$	$32 \leq x \leq 159$
($x = 15, n = 16$)	$15 \leq x \leq 151$	$63 \leq x \leq 103$	$0 \leq x \leq 127$

Table 3: Output of PRORP on Sum0 ($\epsilon_1 = \epsilon_2 = 0.02, k = 16$)

5.4 RQ4: Performance comparison with baseline

We evaluate the precision of PRORP against a baseline of naïve Monte Carlo (MC) sampling on a family of 32-bit instances of the benchmark program Unif2 (Figure 6), which returns uniformly distributed values over a configurable interval.

In this family, the reachable set consists of offsets $x \in [x_0, (x_0 + 2^{32-\ell}) \bmod 2^{32})$ for $\ell = 0, \dots, 32$, with precondition $\varphi := (x = x_0)$. We take parameters $x_0 = 2^{51}$ and $\epsilon_1 = \epsilon_2 = 0.05, \delta = 0.1, k = 32$, and report *Fo-error* and *Ba-error* (as percentages) for each ℓ . Figure 7 plots three curves: MC Fo-error, PRORP Fo-error, and PRORP Ba-error (Ba-error is always zero for MC).

In the *large-support regime* ($\ell \leq 18$), MC yields essentially 100% Fo-error as the sampling budget is overwhelmed by the support size, whereas PRORP attains near-zero Fo and Ba errors. In the *transition regime* ($18 \leq \ell \leq 22$), MC Fo-error drops sharply while PRORP incurs minor errors due to tree generalization yet remains substantially more accurate. In the *small-support regime* ($\ell \geq 23$), both methods achieve negligible error. Overall, PRORP maintains low error across all support sizes, whereas MC becomes competitive only once the reachable support is small enough for exhaustive sampling.

6 Related Work

There has been prior work on analysis of probabilistic programs. Exist [Bao *et al.*, 2024], Prinsys [Gretz *et al.*, 2013], Chen *et al.* [Chen *et al.*, 2015], Feng *et al.* [Feng *et al.*, 2017], Wang *et al.* [Wang *et al.*, 2018], MORA [Bartocci *et al.*, 2020], and CegisPro2 [Batz *et al.*, 2023] perform quantitative invariant synthesis using diverse techniques like template-based synthesis, abstract interpretation and data-driven or counterexample-guided synthesis. PSI [Gehr *et al.*, 2016] and λ PSI [Gehr *et al.*, 2020] compute posterior distributions via computer algebra, while PSE [Geldenhuys *et al.*, 2012] and Plinko [Susag *et al.*, 2022] enable symbolic execution of randomized programs. However, unlike our work, these techniques synthesize inductive invariants for all reachable states.

Our objective is related to prior work on angelic verification [Das *et al.*, 2015; Das and Lal, 2017; Joshi *et al.*, 2012]. The set of *all* reachable states computed by “regular” verifiers tends to be pessimistic, as many error paths stem from rare environmental behaviors. Angelic verification mitigates pessimistic overapproximation by favoring “benign” environment behaviors, helping reduce false positives. Our approach instead models the environment probabilistically and learns symbolic summaries of statistically likely states—for example, while occasional network failures may occur, long sequences of failures should remain provably improbable, with probabilities estimated from data or domain expertise.

Our methodology follows the classic *guess-and-check* paradigm, where candidate solutions are iteratively generated and verified. Generation strategies range from decision trees [Aiken *et al.*, 2016; D’Souza *et al.*, 2018; Garg *et al.*, 2014; 2016; Riley and Fedyukovich, 2022], PAC learning [Sharma *et al.*, 2013], reinforcement learning [Naik, 2018; Yu *et al.*, 2023], and continuous logic networks [Ruan *et al.*, 2019;

Yao *et al.*, 2020], to more recent approaches using large language models [Chakraborty *et al.*, 2023; Wu and others, 2024]. However, these techniques produce inductive invariants for *all* reachable states, whereas PRORP produces symbolic summaries of *statistically likely* reachable states with finite-sample approximation guarantees.

7 Proof of hardness of $\epsilon\text{-Cl}_k(\varphi)$ Emptiness problem

We begin by defining a key decision problem in our setting.

Problem 1 ($\epsilon\text{-Cl}_k(\varphi)$ Emptiness). *Given a probabilistic Boolean program P with input variables $\mathcal{X}$, a distribution $\mathcal{D}$ over initial states, a precondition $\varphi \subseteq \Omega_P$, a step bound $k \in \mathbb{N}$, and a threshold $\epsilon \in (0, 1]$, decide whether there exists a state $\sigma \in \epsilon\text{-Cl}_k(\varphi)$, i.e., whether there exists a state reachable from R_φ within k steps with probability at least ϵ .*

The corresponding synthesis problem of computing $\epsilon\text{-Cl}_k(\varphi)$ is at least as hard, since emptiness can be decided by testing whether the synthesized set is empty.

Theorem 3. *Problem 1 ($\epsilon\text{-Cl}_k(\varphi)$ Emptiness) is PSPACE-complete.*

To prove Theorem 3, we establish two claims.

Claim (a): Problem 1 is PSPACE-hard.

Claim (b): Problem 1 lies in PSPACE; i.e., it can be decided by a PSPACE algorithm.

We first prove Claim (a) via a polynomial-time Karp reduction from the known PSPACE-complete problem **DFA-INTERSECTION** [Kozen, 1977]: Given a finite set $\{A_i\}_{i=1}^l$ of DFAs over a common alphabet Σ , decide whether there exists a word $w \in \Sigma^*$ such that $w \in \bigcap_{i=1}^l L(A_i)$.

7.1 Reduction Procedure (Proving PSPACE-Hardness)

We are given a finite collection $\{A_i\}_{i=1}^l$ of l DFAs defined over the common alphabet set Σ - $A_i = (Q_i, \Sigma, \delta_i, \bar{q}_i, F_i)$, where for all $i \in [l]$, $|Q_i| \leq n$ and $|\Sigma| = m$. We will use Lemma 1 in order to provide the reduction.

Lemma 1. *If the intersection language $L_{int} = \bigcap_{i=1}^l L(A_i)$ of a finite collection $\{A_i\}_{i=1}^l$ of l DFAs defined over the common alphabet set Σ is nonempty, then there exists a word $w \in \Sigma^{\leq n^l}$ such that $w \in L_{int}$.*

Proof. In order to check if the intersection language $\bigcap_{i=1}^l L(A_i)$ is nonempty, naively we construct a product automaton $A_\otimes = (Q_\otimes, \Sigma, \delta_\otimes, \bar{q}_\otimes, F_\otimes)$ such that

- $Q_\otimes = (Q_1 \times Q_2 \times \dots \times Q_l)$ and $\bar{q}_\otimes = (\bar{q}_1, \bar{q}_2, \dots, \bar{q}_l)$.
- $\delta_\otimes((q_1, q_2, \dots, q_l), a) = (\delta_1(q_1, a), \delta_2(q_2, a), \dots, \delta_l(q_l, a))$.
- $F_\otimes = (F_1 \times F_2 \times \dots \times F_l)$.

Thus a word $w \in \Sigma^*$ is accepted by the product automaton $A_\otimes$ if and only if it is accepted by each of the DFAs $\{A_i\}_{i=1}^l$, or in other words, $w \in L(A_\otimes) \iff w \in \bigcap_{i=1}^l L(A_i)$. Suppose the language intersection is non-empty, i.e., $L(A_\otimes) \neq \emptyset$. This implies that there exists at least one word w such that $w \in L(M_{prod})$.

Let w be a word of minimal length in $L(A_\otimes)$. Let the sequence of states traversed by the product automaton upon reading the word $w = w_1w_2 \cdots w_m$ be

$$\bar{q}_\otimes = s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \rightarrow \cdots \xrightarrow{a_m} s_m = \bar{q}_{acc},$$

where $\bar{q}_{acc} \in F_\otimes$. The length of this path is $m = |w|$. The number of states visited in this path is $m + 1$. We will prove by contradiction that $|w| \leq |Q_{prod}| = n^l$. Assume, for the sake of contradiction, that $|w| > n^l$. The number of states in the traversed sequence is $m + 1 > n^l$. By the pigeonhole principle, since there are more states in the sequence than there are unique states in $Q_\otimes$, at least one state must be repeated. That is, there must exist indices r and t with $0 \leq r < t \leq m$ such that $s_r = s_t$. This repeated state indicates the presence of a cycle in the state traversal path. Let $w = uxv$, where u is the prefix corresponding to the path from s_0 to $s_r = s_t$, x is the substring corresponding to the path from s_r back to itself, and v is the suffix corresponding to the path from s_t to the final state s_m . The length of v is $|v| = t - r \geq 1$.

Consider the word $w' = uv$. This word is formed by removing the subword x from w . Since s_m is a final state, the word w' is also accepted by $A_\otimes$. The length of w' is $|w'| = |u| + |v| = |u| + |v| + |x| - |x| = |w| - |x|$. Since $|x| \geq 1$, we have $|w'| < |w|$. This contradicts our initial assumption that w was a word of minimal length in $L(M_{prod})$. Therefore, our assumption that $|w| > n^l$ must be false. It follows that $|w| \leq n^l$. This proves that if the language intersection is non-empty, a witness word of length at most n^l must exist. $\square$

Reduction Construction. We construct a probabilistic Boolean program PP_A that simulates a bounded execution of $A_\otimes$. The program maintains an array of control variables

$$\text{state}[i] \in Q_i \quad \text{for } i \in [\ell],$$

encoded using $\lceil \log_2 n \rceil$ bits per component. The initial configuration is

$$\varphi \equiv \bigwedge_{i=1}^{\ell} (\text{state}[i] = q_i^0).$$

We set the step bound $k = n^\ell$ and threshold $\epsilon = 0$.

Algorithm 5 Probabilistic Program PP_A (Template)

```

1: Input: DFAs  $\{A_i\}_{i=1}^l$ 
2: Initialize  $\epsilon = 0$ ,  $k = n^\ell$ ,  $\text{iters} \leftarrow 0$ 
3: for  $i = 1$  to  $l$  do
4:    $\text{state}[i] \leftarrow \bar{q}_i$ 
5: while  $\neg(\bigcap_{j=1}^l \text{state}[j] \in F_j)$  and  $\text{iters} < k$  do
6:    $\text{symbol} \leftarrow$  uniformly sample from  $\Sigma$ 
7:   for  $i = 1$  to  $l$  do
8:      $\text{state}[i] \leftarrow \delta_i(\text{state}[i], \text{symbol})$ 
9:    $\text{iters} \leftarrow \text{iters} + 1$ 
10: if  $\bigcap_{j=1}^l \text{state}[j] \in F_j$  then
11:   return NONEMPTY
12: else
13:   return EMPTY

```

In Algorithm 5, the probabilistic program PP_A encodes the emptiness of the intersection of DFAs as a bounded stochastic exploration of their joint state space. Given DFAs $\{A_i\}_{i=1}^l$, each defined as $A_i = (Q_i, \Sigma, \delta_i, \bar{q}_i, F_i)$, the program first initializes its parameters in Algorithm 5, setting $\epsilon = 0$, the exploration bound $k = n^\ell$ (corresponding to the number of states in the product automaton), and $\text{iters} \leftarrow 0$.

Each DFA begins in its start state $\bar{q}_i$ as assigned in Algorithm 5, thereby forming the initial product configuration $(\text{state}[1], \dots, \text{state}[\ell])$. The main exploration loop in Algorithm 5 continues as long as no accepting configuration is reached and the iteration count remains below k . At each iteration, a symbol is uniformly sampled from the alphabet Σ (Algorithm 5), and each DFA transitions to its next state using its transition function δ_i (Algorithm 5). The iteration counter is incremented in Algorithm 5 to track the current unroll depth. When the loop terminates, the final configuration is checked in Algorithm 5: if all DFAs are simultaneously in accepting states, the procedure outputs **NONEMPTY** (Algorithm 5); otherwise, it outputs **EMPTY** (Algorithm 5), indicating that no common accepted word was found within the exploration bound. Overall, PP_A performs a finite-depth probabilistic search over the product automaton, conclusively detecting nonemptiness when an accepting configuration is reached and approximating emptiness otherwise.

Correctness of the Reduction. By Lemma 1, if $\exists w \in \bigcap_i L(A_i)$, then $|w| \leq n^\ell$. Hence, under the precondition $\varphi = (\bigcap_j \text{state}[j] = \bar{q}_j, \text{iters} = 0)$, the program reaches a final accepting configuration within $k = n^\ell$ steps with nonzero probability:

$$\Pr[\text{C1}_k(\varphi) \text{ reached in } n^\ell \text{ steps}] = \frac{1}{|\Sigma|^{n^\ell}} = \frac{1}{m^{n^\ell}} > 0.$$

Therefore, deciding whether $\epsilon\text{-C1}_k(\varphi)$ is empty for $0 \leq \epsilon < \frac{1}{m^{n^\ell}}$ is equivalent to deciding **DFA-INTERSECTION**, establishing PSPACE-hardness.

7.2 PSPACE Membership

Having established PSPACE-hardness, we now prove the matching upper bound by showing that $\epsilon\text{-C1}_k(\varphi)$ emptiness lies in PSPACE. The key observation is that emptiness reduces to bounded reachability in the synchronous product automaton of the DFAs, but explicit construction of the product incurs an exponential blow-up. Instead, we employ a symbolic recursive reachability procedure in the style of Savitch [Savitch, 1970], thereby avoiding materialization of the product automaton.

Symbolic Reachability. Let $A_i = (Q_i, \Sigma, \delta_i, q_i^0, F_i)$ for $i \in [\ell]$ be the DFAs from the hardness proof with $|Q_i| \leq n$. The synchronous product automaton

$$A_\otimes = (Q_\otimes, \Sigma, \delta_\otimes, q_\otimes^0, F_\otimes)$$

is defined as usual by

$$Q_\otimes = Q_1 \times \dots \times Q_\ell, q_\otimes^0 = (q_1^0, \dots, q_\ell^0), F_\otimes = F_1 \times \dots \times F_\ell,$$

and

$$\delta_\otimes((q_1, \dots, q_\ell), a) = (\delta_1(q_1, a), \dots, \delta_\ell(q_\ell, a)).$$

As before, the bounded witness length is $k = n^\ell$. Thus $\epsilon\text{-C1}_k(\varphi)$ emptiness reduces to deciding whether some $\sigma \in F_\otimes$ is reachable from $q_\otimes^0$ by a path of length at most k in $A_\otimes$.

We now describe the decision algorithm.

Algorithm 6 DET- PP_A : PSPACE Decision Procedure for $\epsilon\text{-Cl}_k(\varphi)$ Emptiness

```
1: procedure REACH( $s, t, m$ )
2:   if  $m = 0$  then
3:     return ( $s = t$ )
4:   else if  $m = 1$  then
5:     for  $a \in \Sigma$  do
6:       if  $\delta_{\otimes}(s, a) = t$  then
7:         return TRUE
8:     return FALSE
9:   else
10:     $\text{mid} \leftarrow \lfloor m/2 \rfloor$ 
11:    for  $u \in Q_{\otimes}$  do
12:      if REACH( $s, u, \text{mid}$ ) and REACH( $u, t, m - \text{mid}$ ) then
13:        return TRUE
14:    return FALSE
15:   $q_{\otimes}^0 \leftarrow (q_1^0, \dots, q_{\ell}^0)$ 
16:  for  $\sigma \in F_{\otimes}$  do
17:    if REACH( $q_{\otimes}^0, \sigma, k$ ) then
18:      ACCEPT
19: REJECT
```

Correctness. For $m = 0$, REACH($s, t, 0$) returns **TRUE** iff $s = t$, which is correct. For $m = 1$, the procedure checks whether there exists a symbol $a \in \Sigma$ such that $\delta_{\otimes}(s, a) = t$, exactly matching the definition of a length-1 path. For $m > 1$, the procedure returns **TRUE** iff there exists some $u \in Q_{\otimes}$ such that u is reachable from s in $\lfloor m/2 \rfloor$ steps and t is reachable from u in $m - \lfloor m/2 \rfloor$ steps. This is equivalent to the existence of a length- m path by splitting it at the midpoint. Thus REACH(s, t, m) returns **TRUE** iff there exists a path of length m from s to t in $A_{\otimes}$.

The outer loop checks whether any accepting state $\sigma \in F_{\otimes}$ is reachable from $q_{\otimes}^0$ within k steps. Since emptiness holds precisely when no such σ is reachable, the algorithm decides $\epsilon\text{-Cl}_k(\varphi)$ emptiness.

Space Complexity. We analyze the space usage of DET- PP_A . A product state $s \in Q_{\otimes}$ is a tuple of ℓ components, each requiring $\lceil \log_2 n \rceil$ bits, so storing s requires $O(\ell \log n)$ bits. Each call to REACH stores two states and a counter $m \leq k$, so each frame uses

$$O(\ell \log n + \log k) = O(\ell \log n + \ell \log n) = O(\ell \log n)$$

bits, since $\log k = \log(n^{\ell}) = \ell \log n$.

The recursion depth is $O(\log m) = O(\log k) = O(\ell \log n)$, so the total space is

$$O((\ell \log n) \cdot (\ell \log n)) = O(\ell^2 \log^2 n),$$

which is polynomial in the input size. No exponential structure is constructed explicitly, and no frame stores more than two product states at once. Hence the algorithm runs in polynomial space.

Theorem 4. *The $\epsilon\text{-Cl}_k(\varphi)$ emptiness problem lies in PSPACE.*

Combining the two claims, we conclude that $\epsilon\text{-Cl}_k(\varphi)$ **Emptiness is PSPACE-complete.**

8 Correctness of PRORP

Before we go into the details of the algorithm, we'll be using the following additive Chernoff bound in our proofs.

Theorem 5 (Additive Chernoff bound). *Let $X_1, X_2, \dots, X_n$ be independent and Bernoulli random variables, i.e., $X_i \in \{0, 1\}$, and let*

$$X := \frac{1}{n} \sum_{i=1}^n X_i, \quad \mu := \mathbb{E}[X] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}[X_i].$$

Then, for any $\varepsilon > 0$, we have

$$\Pr[|X - \mu| \geq \varepsilon] \leq 2 \exp(-2n\varepsilon^2),$$

Lemma 2 (Well-definedness of the Induced Output Distribution). *For any probabilistic program P , precondition φ , input distribution $\mathcal{D}$ over R_φ , and loop bound k , the induced output distribution $\mathcal{D}_{P,\varphi}^k$ is a well-defined probability distribution over terminal states.*

Proof. Let X_0 be a random variable ranging over R_φ with distribution $\mathcal{D}$. For a fixed initial state $\sigma \in R_\varphi$, execution of P for at most k steps induces a probability distribution over terminal states, since P has a finite branching factor at each step and all probabilistic choices are governed by well-defined probability distributions.

Define the random variable Y as the terminal state reached by executing P for at most k steps starting from X_0 . Then Y is a measurable function of the sequence of probabilistic choices made during execution.

For any terminal state σ' , define

$$\begin{aligned} \mathcal{D}_{P,\varphi}^k(\sigma') &:= \Pr[Y = \sigma'] \\ &= \sum_{\sigma \in R_\varphi} \mathcal{D}(\sigma) \cdot \Pr\left[P \text{ reaches } \sigma' \text{ from } \sigma \text{ within } k \text{ steps}\right]. \end{aligned}$$

Since $\mathcal{D}$ is a probability distribution over R_φ and, for each σ , the probabilities of reaching terminal states within k steps sum to 1, it follows that

$$\sum_{\sigma'} \mathcal{D}_{P,\varphi}^k(\sigma') = 1.$$

Hence, $\mathcal{D}_{P,\varphi}^k$ is a well-defined probability distribution over terminal states. $\square$

Lemma 3 (Correctness of FoRT). *Let P be a probabilistic program, φ a precondition, $\mathcal{D}$ an input distribution over R_φ , and $k \in \mathbb{N}$ a loop-unrolling bound. Let ψ be a candidate invariant expressed as a DNF, and let $\epsilon, \delta \in (0, 1]$.*

Let $\text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi))$ denote the probability mass of terminal states reachable from R_φ within k steps that violate ψ .

Then FoRT, when run with

$$m = \left\lceil \frac{2}{\epsilon^2} \log\left(\frac{4}{\delta}\right) \right\rceil$$

samples, outputs an estimate Error_1 such that

$$\Pr\left[|\text{Error}_1 - \text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi))| \leq \frac{\epsilon}{2}\right] \geq 1 - \delta/2. \quad (1)$$

Proof. We fix P , φ , $\mathcal{D}$, k , and ψ . Let $\mathcal{D}_{P,\varphi}^k$ denote the induced output distribution over terminal states obtained by sampling an initial state from $\mathcal{D}$ and executing P for at most k steps.

ForT samples m initial states

$$X_1, X_2, \dots, X_m \sim \mathcal{D}$$

independently. For each $i \in [m]$, the program P is executed independently from X_i for at most k steps, yielding a terminal state Y_i . By construction, the random variables $Y_1, \dots, Y_m$ are independent and identically distributed according to $\mathcal{D}_{P,\varphi}^k$.

For each $i \in [m]$, define the indicator random variable

$$Z_i := \mathbf{1}\{Y_i \in \text{Cl}_k(\varphi) \setminus R_\psi\}.$$

Thus, $Z_i = 1$ if and only if the i -th execution produces a terminal state that is reachable from R_φ within k steps and violates ψ .

ForT computes the empirical mean

$$\text{Error}_1 := \frac{1}{m} \sum_{i=1}^m Z_i.$$

Since each Y_i is distributed according to $\mathcal{D}_{P,\varphi}^k$,

$$\mathbb{E}[Z_i] = \Pr_{Y \sim \mathcal{D}_{P,\varphi}^k} [Y \in \text{Cl}_k(\varphi) \setminus R_\psi] = \text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi)).$$

By linearity of expectation,

$$\mathbb{E}[\text{Error}_1] = \text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi)),$$

and hence Error_1 is an unbiased estimator of $\text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi))$.

The random variables $Z_1, \dots, Z_m$ are independent Bernoulli random variables. Applying an additive Chernoff bound yields, for any $\varepsilon > 0$,

$$\Pr[|\text{Error}_1 - \mathbb{E}[\text{Error}_1]| \geq \varepsilon] \leq 2 \exp(-2m\varepsilon^2).$$

Setting $\varepsilon = \epsilon/2$ gives

$$\Pr\left[|\text{Error}_1 - \text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi))| \geq \frac{\epsilon}{2}\right] \leq 2 \exp\left(-\frac{m\epsilon^2}{2}\right).$$

With

$$m = \left\lceil \frac{2}{\epsilon^2} \log\left(\frac{4}{\delta}\right) \right\rceil,$$

we obtain

$$2 \exp\left(-\frac{m\epsilon^2}{2}\right) \leq \delta/2.$$

Therefore,

$$\Pr\left[|\text{Error}_1 - \text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi))| \leq \frac{\epsilon}{2}\right] \geq 1 - \delta/2,$$

which completes the proof. $\square$

Lemma 4. *Let P be a probabilistic program, φ a precondition, and $k \in \mathbb{N}$ a loop-unrolling bound. Let ψ be a candidate invariant expressed as a DNF formula, and let $\epsilon, \delta \in (0, 1]$. Let $\text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi))$ denote the probability, under the uniform distribution over R_ψ , that a sampled state is not reachable from R_φ within k steps.*

Then BaRT, when run with

$$m = \left\lceil \frac{2}{\epsilon^2} \log\left(\frac{4}{\delta}\right) \right\rceil$$

samples, outputs an estimate Error_2 such that

$$\Pr\left[|\text{Error}_2 - \text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi))| \leq \frac{\epsilon}{2}\right] \geq 1 - \delta/2. \quad (2)$$

Proof. We fix P , φ , k , and ψ .

BaRT samples m states

$$S_1, S_2, \dots, S_m \sim \mathcal{U}(R_\psi)$$

independently using the almost-uniform sampler `Pepin`. The randomness used by `Pepin` is independent across samples.

For each sampled state S_i , BaRT constructs a Boolean formula Υ_i encoding reachability of S_i from R_φ within k steps, namely:

$$\Upsilon_i := \varphi(X_0) \wedge \bigwedge_{j=0}^{k-1} F_P(X_j, X_{j+1}) \wedge \bigvee_{j=1}^k (X_j = S_i).$$

By construction of the program transition formula F_P , the formula Υ_i is satisfiable if and only if there exists a valid execution of P of length at most k starting from some initial state in R_φ and terminating in state S_i . Consequently,

$$\Upsilon_i \text{ is unsatisfiable} \iff S_i \notin \text{Cl}_k(\varphi).$$

For each $i \in [m]$, define the indicator random variable

$$Z_i := \mathbf{1}\{S_i \notin \text{Cl}_k(\varphi)\}.$$

Thus, $Z_i = 1$ if and only if the i -th sampled state violates reachability.

BaRT computes the empirical mean

$$\text{Error}_2 := \frac{1}{m} \sum_{i=1}^m Z_i.$$

Since each S_i is sampled independently from $\mathcal{U}(R_\psi)$,

$$\mathbb{E}[Z_i] = \Pr_{S \sim \mathcal{U}(R_\psi)}[S \notin \text{Cl}_k(\varphi)] = \text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi)).$$

By linearity of expectation,

$$\mathbb{E}[\text{Error}_2] = \text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi)).$$

Hence, Error_2 is an unbiased estimator of $\text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi))$.

The random variables $Z_1, \dots, Z_m$ are mutually independent since: (i) the samples $S_1, \dots, S_m$ are independent by construction of `Pepin`, and (ii) each reachability check Υ_i is a deterministic function of S_i .

The variables $Z_1, \dots, Z_m$ are independent Bernoulli random variables. Applying an additive Chernoff (Hoeffding) bound, for any $\varepsilon > 0$,

$$\Pr[|\text{Error}_2 - \mathbb{E}[\text{Error}_2]| \geq \varepsilon] \leq 2 \exp(-2m\varepsilon^2).$$

Setting $\varepsilon = \epsilon/2$ yields

$$\Pr\left[|\text{Error}_2 - \text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi))| \geq \frac{\epsilon}{2}\right] \leq 2 \exp\left(-\frac{m\epsilon^2}{2}\right).$$

With

$$m = \left\lceil \frac{2}{\epsilon^2} \log\left(\frac{4}{\delta}\right) \right\rceil,$$

we obtain

$$2 \exp\left(-\frac{m\epsilon^2}{2}\right) \leq \delta/2.$$

Therefore,

$$\Pr\left[|\text{Error}_2 - \text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi))| \leq \frac{\epsilon}{2}\right] \geq 1 - \delta/2,$$

which completes the proof. $\square$

Theorem 6 (Correctness of PRORP). *Given a probabilistic program P , a precondition φ , the number of iterations k , Fo and Ba error parameters $\epsilon, \epsilon_2 \in (0, 1]$, confidence parameter $\delta \in (0, 1]$, PRORP outputs a candidate DNF formula ψ representing a (ϵ, ϵ_2) -approximate statistically likely k -step reachable set with the following theoretical guarantees:*

- *If the algorithm PRORP halts before timeout, it outputs a candidate ψ such that with probability atleast $1 - \delta$, both $\text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi)) < \epsilon_2$ and $\text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi)) < \epsilon$ holds.*
- *In case of a timeout, PRORP does not provide any theoretical guarantees on the outputted candidate ψ .*

Proof. From the design of the algorithm PRORP, we know that if it halts before a user-specified time-out, it outputs a candidate ψ which is ‘approved’ by both FoRT and BaRT, i.e., if the Fo-error and Ba-error estimates Error_1 and Error_2 satisfy the following conditions :-

$$\text{Error}_1 \leq \epsilon_1/2$$

and

$$\text{Error}_2 \leq \epsilon_2/2$$

The theoretical guarantees provided by FoRT (Equation 1) and BaRT (Equation 2) states the following:

$$\Pr[|\text{Error}_1 - \text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi))| \leq \epsilon_1/2] \geq 1 - \delta/2 \quad (3)$$

and

$$\Pr[|\text{Error}_2 - \text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi))| \leq \epsilon_2/2] \geq 1 - \delta/2 \quad (4)$$

From equation 3, we obtain the following with probability atleast $1 - \delta/2$,

$$|\text{Error}_1 - \text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi))| \leq \epsilon_1/2 \quad (5)$$

Table 4: Quality of PRORP on LinExp

R_φ	$\text{Cl}_k(\varphi)$	$\epsilon\text{-Cl}_k(\varphi)$	R_ψ
(count = 27, n = 8)	$[43, 51] \times [0]$	$[44, 50] \times [0]$	$[44, 51] \times [0]$
(count = 27, n = 16)	$[59, 75] \times [0]$	$[63, 72] \times [0]$	$[60, 79] \times [0]$
(count = 27, n = 32)	$[91, 123] \times [0]$	$[102, 113] \times [0]$	$[96, 123] \times [0]$
(count = 27, n = 64)	$[155, 219] \times [0]$	$[180, 194] \times [0]$	$[160, 216] \times [0]$

Since the first condition for PRORP to halt before being timed out is $\text{Error}_1 \leq \epsilon_1/2$, with probability atleast $1 - \delta/2$, we have $\text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi)) \leq \epsilon_1$. Similarly, from equation 4, we obtain the following with probability atleast $1 - \delta/2$,

$$|\text{Error}_2 - \text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi))| \leq \epsilon_2/2 \quad (6)$$

The second condition for PRORP to halt before being timed out is given by $\text{Error}_2 \leq \epsilon_2/2$. Thus, with probability atleast $1 - \delta$, we have $\text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi)) \leq \epsilon_2$. Now, since the events 5 and 6 are independent, by combining equations 4 and 3, we obtain the following -

$$\begin{aligned} E_1 &:= \{|\text{Error}_1 - \text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi))| \leq \epsilon_1/2\}, \\ E_2 &:= \{|\text{Error}_2 - \text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi))| \leq \epsilon_2/2\}. \end{aligned}$$

$$\Pr[E_1 \cap E_2] \geq 1 - \delta. \quad (7)$$

Thus, if and when PRORP halts, it outputs a candidate ψ such that with probability atleast $1 - \delta$, $\text{Error}_{\text{Fo}}(\psi, \text{Cl}_k(\varphi)) < \epsilon_1$ and $\text{Error}_{\text{Ba}}(\psi, \text{Cl}_k(\varphi)) < \epsilon_2$. □

9 Case studies on probabilistic programs

LinExp.

The program LinExp (see Figure 9) also has a loop whose number of iterations is fixed by the input. The program takes two unsigned integer variables as input, `count` and `n`. If observed carefully, one notices that in each iteration the value of `count` is incremented by 2 with probability $\frac{3}{8}$ and is incremented by 3 with probability $\frac{5}{8}$. Thus a possible significant difference in the value of `count` on termination will only be visible after a lot of iterations. Thus we expect for smaller values of `n`, the $\epsilon\text{-Cl}_k(\varphi)$ to be very similar to $\text{Cl}_k(\varphi)$, but for larger values of `n` these two sets will divulge a bit and we expect this will be illustrated by the output of PRORP.

RevBin [Kaminski and Katoen, 2017].

The loop in program RevBin (Figure 8) decrements `x` only when a fair Bernoulli trial succeeds. Suppose we start from a precondition (`x` = x_0 , `z` = 0), then termination occurs when `x` has been decremented to zero (after x_0 successes). The variable `z` denotes the total number of iterations until termination. Therefore, the random variable Z (the final value of `z`) follows a negative binomial distribution $Z \sim \text{NB}(x_0, \frac{1}{2})$ with $\Pr[Z = t] = \binom{t-1}{x_0-1} \frac{1}{2^t}$ for $t \geq x_0$. Also, $\mathbb{E}[Z] = \text{Var}(Z) = 2x_0$. If we take $x_0 = \frac{k}{16}$ and $p = \frac{1}{2}$, then the distribution is unimodal and approximately bell-shaped (right-skewed), concentrating near $\frac{k}{8}$, with negligible mass in the tails (see Figure 10). As we execute the program k times, the k -step reachable set $\text{Cl}_k(\varphi)$ is $\{(\mathbf{x} = 0, x_0 \leq \mathbf{z} \leq k)\}$. For $k = 64$ and precondition $\varphi := (\mathbf{x} = 4, \mathbf{z} = 0)$, the set $\text{Cl}_k(\varphi) :=$

```
def RevBin(int x, int z):
    while (x - 1 >= 0):
        d = bernoulli(0.5)
        if(d):
            x = x - 1
            z = z + 1
    return {z, x}
```

Figure 8: Program RevBin

```
def LinExp(int n, int count):
    while (n > 0):
        x1 = bernoulli(0.5)
        x2 = bernoulli(0.5)
        x3 = bernoulli(0.5)
        n = n - 1
        c1 = x1 or x2 or x3
        c2 = (not x1) or x2 or x3
        c3 = x1 or (not x2) or x3
        count = count + c1 + c2 + c3
    return {count, n}
```

Figure 9: Program LinExp

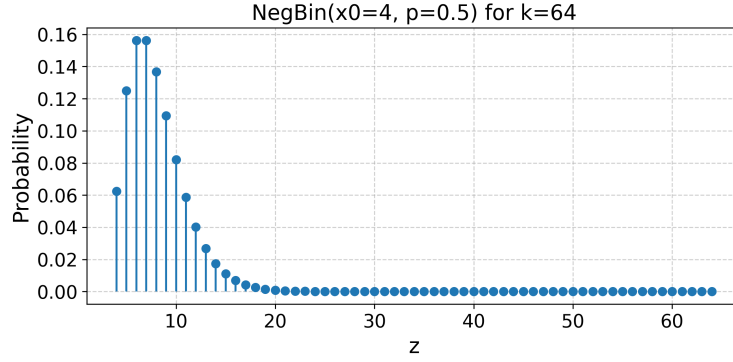


Figure 10: Distribution of loop iterations until termination for RevBin

$\{(x = 0, 4 \leq z \leq 64)\}$. In this case, given $\epsilon_1 = \epsilon_2 = 0.05$ and $\delta = 0.1$, PRORP returns the candidate set $R_\psi = \{(x = 0, 4 \leq z \leq 32)\}$. From Figure 10, we observe that $\epsilon_1\text{-Cl}_k(\varphi) = \{(x = 0, 4 \leq z \leq 11)\}$. Thus, $\epsilon_1\text{-Cl}_k(\varphi) \subset R_\psi \subset \text{Cl}_k(\varphi)$.

10 Experimental results for RQ1

Table 5 contain the complete results for programs modelled using 16-bit bitvectors while Table 6 contain the complete results for programs modelled using 32-bit bitvectors for all the 16 named benchmarks adopted from the experimental setup of [Bao *et al.*, 2024]. These tables contain program name (**Prog**), precondition (**Pre**), unroll depth (**k**), height of the tree (**H**), number of split operations (**#S**), number of prune operations (**#P**), number of learning phases (**PH**), total time in seconds (**TT**) and the final candidate learned (**Candidate**).

Table 5: Experimental Results (16-bit)

Prog	Pre	k	(H,#S,#P)	PH	TT	Candidate
Geo0	z_{14}	16	(2, 0, 0)	1	4.50	z_{14}
		32	(2, 0, 0)	1	4.23	z_{14}
		64	(2, 0, 0)	1	4.52	z_{14}
		128	(2, 0, 0)	1	4.68	z_{14}
Geo0	z_6	16	(6, 0, 0)	1	4.46	$(z_6 \vee \neg z_3) \wedge (z_6 \vee \neg z_4) \wedge (z_6 \vee \neg z_5)$
		32	(5, 0, 0)	1	4.71	$(z_6 \vee \neg z_4) \wedge (z_6 \vee \neg z_5)$
		64	(4, 0, 0)	1	5.15	$(z_6 \vee \neg z_3)$
		128	(2, 0, 0)	1	5.68	True
Geo1	$((z = 0, x = 16))$	16	(33, 153, 0)	12	83.33	$(x, z) \in \{(16.2^i, i) \mid i \in [0, 11]\}$
		32	(33, 40, 0)	6	35.25	$(x, z) \in \{(16.2^i, i) \mid i \in [0, 4]\}$
		64	(33, 48, 0)	7	46.12	$(x, z) \in \{(16.2^i, i) \mid i \in [0, 4]\}$
		128	(33, 157, 0)	8	101.51	$(x, z) \in \{(16.2^i, i) \mid i \in [0, 11]\}$
Geo1	$((z < 4, x = 8))$	16	(33, 47, 0)	6	32.78	$(x, z) \in \{(2^i, j) \mid i \in [3, 4], j \in [0, i]\} \cup \{(2^i, j) \mid i \in [5, 7], j \in [i - 3, i]\}$
		32	(33, 50, 0)	9	50.88	$(x, z) \in \{(2^i, j) \mid i \in [3, 7], j \in [i - 3, i]\}$
		64	(33, 159, 0)	14	110.51	$(x, z) \in \{(0, j) \mid j \in \{16, 18, 48, 50\}\} \cup \{(2^i, j) \mid i \in [3, 7], j \in [i - 3, i]\} \cup \{(2^i, j) \mid i \in [8, 12], j \in [8, 11]\}$
		128	(33, 56, 0)	7	71.47	$(x, z) \in \{(2^i, j) \mid i \in [3, 6], j \in [i - 3, i + 1]\} \cup \{(128, j) \mid j \in [4, 7]\}$
Geo2	$(z = 0, x = 16)$	16	(33, 38, 0)	6	31.32	$(x, z) \in \{(16 + i, i) \mid i \in [0, 7]\}$
		32	(33, 71, 0)	12	66.19	$(x, z) \in \{(16 + i, i) \mid i \in [0, 11]\}$
		64	(33, 93, 0)	7	56.62	$(x, z) \in \{(16 + i, i) \mid i \in [0, 8] \cup \{10, 14\}\}$
		128	(33, 68, 0)	43	353.89	$(x, z) \in \{(16 + i, i) \mid i \in [0, 7] \cup \{18\}\}$
Geo2	$(z < 64, x < 16)$	16	(26, 30, 0)	6	32.19	$(x, z) \in \{[(0 - 19), (0 - 31)], [(20 - 31), (32 - 63)]\}$
		32	(25, 16, 0)	5	26.58	$(x, z) \in \{[(0 - 23), (0 - 63)], [(24 - 31), (64 - 71)]\}$
		64	(31, 28, 0)	7	51.45	$(x, z) \in \{[(0 - 23), (0 - 63)], [(24 - 31), (64 - 71)]\}$
		128	(26, 29, 0)	6	266.25	$(x, z) \in \{[(0 - 23), (0 - 63)], [(0 - 31), (2 - 7, 62 - 69)]\}$
LinExp	$(\text{count} = 0)$	16	(31, 15, 0)	8	64.47	$\text{count} \in [2, 48)$
		32	(27, 7, 0)	3	26.68	$\text{count} \in [0, 96)$
		64	(26, 6, 0)	3	37.32	$\text{count} \in [0, 192)$
		128	(25, 5, 0)	4	31.47	$\text{count} \in [0, 384)$
LinExp	$\text{count} < 64$	16	(59, 23, 0)	4	31.47	$\text{count} \in [0, 96)$
		32	(25, 7, 0)	3	27.64	$\text{count} \in [0, 128)$
		64	(25, 8, 0)	3	44.33	$\text{count} \in [0, 256)$
		128	(25, 6, 0)	23 3	123.02	$\text{count} \in [0, 384)$
Detm	$\text{count} = 0$	16	(32, 26, 0)	7	102.63	$\text{count} \in [1, 17)$
		32	(32, 43, 0)	6	170.16	$\text{count} \in [1, 17)$

Prog	Pre	k	(H,#S,#P)	PH	TT	Candidate
Sum0	$(x < 8, n < k)$	16	(25, 5, 0)	3	20.24	$x \in [0, 128]$
		32	(23, 4, 0)	2	16.33	$x \in [0, 512]$
		64	(22, 6, 0)	3	33.67	$x \in [0, 2048]$
		128	(19, 1, 0)	2	69.60	$x \in [0, 8192]$
Sum0	$(x = 64, n < \frac{k}{2})$	16	(27, 9, 0)	4	32.21	$x \in [64, 160] \cup [176, 184]$
		32	(26, 5, 0)	3	22.75	$x \in [64, 512]$
		64	(21, 2, 0)	2	20.41	$x \in [0, 2048]$
		128	(20, 4, 0)	2	67.19	$x \in [0, 8192]$
Fair	$\text{count} < 64$	16	(13, 1, 0)	2	9.85	$\text{count} \in [0, 80]$
		32	(11, 1, 0)	2	10.98	$\text{count} \in [0, 64]$
		64	(10, 0, 0)	1	7.71	$\text{count} \in [0, 128]$
		128	(10, 0, 0)	1	18.88	$\text{count} \in [0, 128]$
Fair	$\text{count} = 0$	16	(17, 8, 0)	4	24.48	$\text{count} \in [1, 4]$
		32	(17, 7, 0)	4	24.90	$\text{count} \in [1, 4] \cup [17, 20]$
		64	(17, 8, 0)	5	35.38	$\text{count} \in [1, 4]$
		128	(15, 5, 0)	3	45.00	$\text{count} \in [1, 4] \cup [17, 20] \cup [33, 36] \cup [49, 52]$
GeoAr	$(x = 0, y = 0)$	16	(33, 40, 0)	7	35.47	$(y, x) \in \{(i, \sum_{j=0}^{i-1} j); i \in \{1-6\}\}$
		32	(33, 46, 1)	6	34.26	$(y, x) \in \{(i, \sum_{j=0}^{i-1} j); i \in \{1-4, 7\}\}$
		64	(33, 73, 1)	8	62.04	$(y, x) \in \{(i, \sum_{j=0}^{i-1} j); i \in \{1-5, 8, 9, 12\}\}$
		128	(33, 56, 0)	6	233.43	$(y, x) \in \{(i, \sum_{j=0}^{i-1} j); i \in \{1-4, 7, 9\}\}$
GeoAr	$(x < 4, y = 8)$	16	(33, 60, 0)	8	46.43	$(y, x) \in \{(9, x) \mid x \in [0, 3]\} \cup \{(10, x) \mid x \in [9, 12]\} \cup \{(11, x) \mid x \in [19, 22]\} \cup \{(12, x) \mid x \in [30, 33]\} \cup \{(13, x) \mid x \in [44, 45]\}$
		32	(33, 64, 0)	9	57.54	$(y, x) \in \{(9, x) \mid x \in [0, 3]\} \cup \{(10, x) \mid x \in [9, 12]\} \cup \{(11, x) \mid x \in [19, 22]\} \cup \{(12, x) \mid x \in [30, 33]\} \cup \{(13, x) \mid x \in [44, 45]\} \cup \{(14, 58)\}$
		64	(33, 159, 0)	8	105.95	$(y, x) \in \{(9, x) \mid x \in [0, 3]\} \cup \{(10, x) \mid x \in [9, 12]\} \cup \{(11, x) \mid x \in [19, 22]\} \cup \{(12, x) \mid x \in [30, 33]\} \cup \{(13, x) \mid x \in [42, 43]\} \cup \{(14, x) \mid x \in [56, 58]\} \cup \{(15, x) \mid x \in [68, 72]\} \cup \{(16, x) \mid x \in [84, 87]\} \cup \{(17, x) \mid x \in [100, 103]\} \cup \{(18, x) \mid x \in [117, 119]\} \cup \{(19, x) \mid x \in [136, 137]\} \cup \{(20, x) \mid x \in [155]\} \cup \{(23, x) \mid x \in [218, 219]\}$
		128	(33, 115, 0)	7	410.36	$(y, x) \in \{(9, x) \mid x \in [0, 3]\} \cup \{(10, x) \mid x \in [9, 12]\} \cup \{(11, x) \mid x \in [19, 22]\} \cup \{(12, x) \mid x \in [30, 32] \cup [44]\} \cup \{(13, x) \mid x \in [32] \cup [42, 44]\} \cup \{(16, x) \mid x \in [84, 87]\} \cup \{(17, x) \mid x \in [100, 103]\} \cup \{(18, x) \mid x \in [117, 119]\} \cup \{(19, x) \mid x \in [135, 138]\} \cup \{(20, x) \mid x \in [154, 155]\} \cup \{(21, x) \mid x \in [174, 176]\}$

Prog	Pre	k	(H,#S,#P)	PH	TT	Candidate
Bin0	$x = 0, y = 2, n = k$	16	(44, 21, 0)	6	29.91	$x = 2k, y = 2, k \in [0, 16]$
		32	(45, 41, 0)	7	43.41	$y = 2, x = 2k, k \in [0, 16]$
		64	(42, 21, 0)	7	48.08	$y = 2, x = 2k, k \in [0, 16]$
		128	(42, 36, 0)	6	73.80	$y = 2, x = 2k, k \in [0, 16]$
Bin0	$(x < 64) \wedge (y = 32) \wedge (n = k)$	16	(43, 23, 0)	7	35.22	$y = 2, x \in [0, 128]$
		32	(41, 20, 1)	6	33.68	$y = 2, x \in [0, 128]$
		64	(42, 39, 1)	7	53.25	$y = 2, x \in [0, 192]$
		128	(40, 17, 1)	6	73.25	$y = 2, x \in [0, 256]$
Bin1	$x = 0, n = (M - k) / 2$	16	(32, 22, 0)	6	31.79	$x = 8$
		32	(32, 21, 0)	5	29.50	$x = 16$
		64	(32, 21, 0)	5	32.69	$x = 32$
		128	(32, 22, 0)	6	64.68	$x = 64$
Bin1	$x < 32, n = (M - k) / 2$	16	(29, 20, 0)	6	32.90	$x \in [8, 40]$
		32	(32, 35, 0)	7	41.80	$x \in [16, 48]$
		64	(27, 15, 0)	5	35.46	$x \in [32, 64]$
		128	(27, 17, 0)	17	111.10	$x \in [64, 96]$
Bin2	$x = 0, y = 8, n < k/2$	16	(45, 25, 0)	7	36.82	$y = 8, x \in [8, 112]$
		32	(43, 43, 0)	7	52.76	$y = 8, x \in [0, 384]$
		64	(39, 32, 0)	6	89.78	$y = 8, x \in [0, 1024]$
		128	(37, 28, 0)	5	755.47	$y = 8, x \in [0, 4096]$
Bin2	$x < 32, y = 8, n < k/2$	16	(45, 23, 0)	10	52.00	$y = 8, x \in [8, 128]$
		32	(42, 39, 0)	6	41.82	$y = 8, x \in [0, 384]$
		64	(38, 17, 0)	6	75.27	$y = 8, x \in [0, 1024]$
		128	(38, 31, 0)	6	338.01	$y = 8, x \in [0, 3072]$
BiasDir	$x = 0, y = 0$	64	(2, 0, 0)	1	1.55	$(x, y) \in \{(0, 1), (1, 0)\}$
	$x = 1, y = 1$	64	(2, 0, 0)	1	1.56	
DepRV	$x = 0, y = 0, n = k$	16	(48, 61, 0)	7	43.39	$(x, y) \in \{(i, 16-i); i \in [5, 15]\}$
		32	(48, 98, 0)	10	73.29	$(x, y) \in \{(i, 32-i); i \in [9, 23] \cup \{25, 28\}\}$
		64	(48, 113, 0)	6	75.40	$(x, y) \in \{(i, 64-i); i \in [18, 39]\}$
		128	(48, 235, 0)	9	339.35	$(x, y) \in \{(28, 68), (i, 128-i); i \in [43, 82]\}$
DepRV	$x = 8, y = 16, n = k$	16	(48, 61, 0)	9	52.65	$(x, y) \in \{(i, 40-i); i \in [9, 19]\}$
		32	(48, 94, 0)	9	68.06	$(x, y) \in \{(i, 56-i); i \in [14, 31] \setminus \{16\}\}$
		64	(48, 181, 0)	14	163.25	$(x, y) \in \{(i, 88-i); i \in [24, 48] \cup \{50, 52, 54, 56\}\}$
		128	(48, 175, 0)	8	334.72	$(x, y) \in \{(i, 16-i); i \in [5, 15]\}$
RevBin	$x < k/2, z = 0$	16	(64, 26, 0)	8	54.92	$z \in [0, 16]$
		32	(59, 23, 0)	3	23.34	$z \in [0, 32]$
		64	(26, 6, 0)	3	24.73	$z \in [0, 64]$
		128	(25, 5, 0)	3	30.27	$z \in [0, 128]$
RevBin	$x = k/2, z < 16$	16	(29, 17, 0)	4	46.57	$z \in [8, 32]$
		32	(28, 10, 0)	4	34.16	$z \in [16, 48]$
		64	(28, 10, 0)	3	35.23	$z \in [48, 80]$
		128	(28, 10, 0)	4	86.03	$z \in [64, 144]$
Mart	$c = 0, b = 64, r = 0$	16	(48, 28, 0)	12	15.14	$c = 64, r \in [1, 8]$
		32	(48, 48, 0)	6	38.66	$c = 64, r \in [1, 16]$
		64	(48, 45, 0)	6	52.53	$c = 64, r \in [0, 24]$
		128	(48, 30, 0)	12	165.12	$c = 64, r \in [1, 16]$
Mart	$c < 32, b = 64, r = 32$	16	(44, 49, 0)	9	51.67	$c \in [64, 96], r \in [33, 48]$
		32	(43, 24, 0)	8	49.52	$c \in [64, 96], r \in [33, 48]$
		64	(43, 60, 0)	7	77.28	$c \in [64, 96], r \in [33, 48]$
		128	(43, 24, 0)	7	222.36	$c \in [64, 96], r \in [33, 48]$

Table 6: Experimental Results (32-bit)

Prog	Pre	k	(H,#S,#P)	PH	TT	Candidate
Geo0	z_{28}	16	(0, 0, 0)	1	4.44	z_{28}
		32	(2, 0, 0)	1	4.63	
		64	(2, 0, 0)	1	5.08	
		128	(2, 0, 0)	1	5.74	
Geo0	z_{20}	16	(6, 0, 0)	1	4.61	z_{20}
		32	(2, 0, 0)	1	4.72	
		64	(2, 0, 0)	1	5.04	
		128	(2, 0, 0)	1	5.74	
Geo1	(z = 0, x = 16)	16	(65, 78, 0)	12	86.83	$(x, z) \in \{(16 \cdot 2^i, i) \mid i \in [0, 4]\}$
		32	(65, 80, 0)	12	95.81	$(x, z) \in \{(16 \cdot 2^i, i) \mid i \in [0, 4]\}$
		64	(65, 87, 0)	13	116.84	$(x, z) \in \{(16 \cdot 2^i, i) \mid i \in [0, 4] \cup \{6\}\}$
		128	(65, 77, 0)	12	154.74	$(x, z) \in \{(16 \cdot 2^i, i) \mid i \in [0, 3] \cup \{5\}\}$
Geo1	(z < 64, x < 16)	16	(65, 103, 0)	14	110.64	$(x, z) \in \{(2^i, j) \mid i \in [3, 7], j \in [i - 3, i]\} \cup \{(48, 5)\} \cup \{(512, j) \mid j \in [6, 9]\}$
		32	(65, 85, 0)	15	110.29	$(x, z) \in \{(2^i, j) \mid i \in [3, 7], j \in [i - 3, i]\} \cup \{(16, 5)\}$
		64	(65, 79, 0)	15	130.99	$(x, z) \in \{(2^i, j) \mid i \in [3, 7], j \in [i - 3, i]\}$
		128	(65, 80, 0)	14	222.92	$(x, z) \in \{(2^i, j) \mid i \in [3, 7], j \in [i - 3, i]\}$
Geo2	(z = 0, x = 16)	16	(65, 72, 0)	16	100.38	$(x, z) \in \{(16 + i, i) \mid i \in [0, 7]\}$
		32	(65, 70, 0)	19	126.21	$(x, z) \in \{(16 + i, i) \mid i \in [0, 7]\}$
		64	(65, 73, 0)	18	104.99	$(x, z) \in \{(16 + i, i) \mid i \in [0, 7]\}$
		128	(65, 134, 0)	57	683.81	$(x, z) \in \{(16 + i, i) \mid i \in [0, 7] \cup \{16\}\}$
Geo2	(z < 64, x < 16)	16	(56, 45, 0)	13	77.27	$(x, z) \in \{[0, 24] \times [0, 64]\}$
		32	(57, 48, 0)	13	91.34	$(x, z) \in \{[0, 32] \times [0, 72]\}$
		64	(57, 48, 0)	13	127.49	$(x, z) \in \{[0, 24] \times [0, 64], [24, 32] \times [64, 72]\}$
		128	(62, 101, 0)	57	683.81	$(x, z) \in \{[0, 32] \times [0, 72], [16, 32] \times [72, 80]\}$
LinExp	count = 0	16	(63, 27, 0)	11	74.47	count $\in [2, 48]$
		32	(59, 25, 0)	8	87.47	count $\in [0, 96]$
		64	(58, 22, 0)	7	170.11	count $\in [0, 192]$
		128	(57, 22, 0)	7	823.76	count $\in [0, 384]$
LinExp	count < 64	16	(59, 23, 0)	7	50.42	count $\in [0, 96]$
		32	(57, 20, 0)	7	78.33	count $\in [0, 128]$
		64	(56, 21, 0)	7	186.28	count $\in [0, 256]$
		128	(57, 22, 0)	7	903.52	count $\in [0, 384]$
Detm	count = 0	16	(64, 57, 0)	14	168.26	x = 16, count $\in [1, 17]$
		32	(64, 58, 0)	14	219.27	
		64	(64, 57, 0)	14	268.56	
Sum0	x < 8, n < k	16	(57, 21, 0)	7	46.61	x $\in [0, 128]$
		32	(55, 20, 0)	6	48.37	x $\in [0, 512]$
		64	(53, 17, 0)	6	75.88	x $\in [0, 2048]$
		128	(51, 15, 0)	5	249.01	x $\in [0, 8192]$
Sum0	x = 64, n < $\frac{k}{2}$	16	(60, 22, 0)	8	51.09	x $\in [64, 168] \cup [176, 184]$
		32	(58, 20, 0)	6	48.05	x $\in [64, 512]$
		64	(53, 19, 0)	6	76.59	x $\in [0, 2048]$
		128	(51, 12, 0)	5	264.85	x $\in [0, 8192]$
Fair	count = 0	16	(34, 25, 0)	7	36.12	count $\in \{1, 2, 3\}$
		32	(33, 24, 0)	7	41.63	
		64	(34, 25, 0)	7	64.21	
		128	(33, 25, 0)	7	173.63	
Fair	count < 64	16	(27, 16, 0)	5	27.26	count $\in [0, 64]$
		32	(27, 16, 0)	6	38.29	count $\in [0, 64]$
		64	(26, 15, 0)	5	61.47	count $\in [0, 128]$
		128	(26, 15, 0)	5	204.10	count $\in [0, 128]$

Prog	Pre	k	(H,#S,#P)	PH	TT	Candidate
GeoAr	$x = 0, y = 0$	16	(65, 78, 0)	12	87.39	$(y, x) \in \{(i, \sum_{j=0}^{i-1} j) \mid i \in \{1-5, 7\}\}$
		32	(65, 84, 0)	15	112.01	$(y, x) \in \{(i, \sum_{j=0}^{i-1} j) \mid i \in \{1-4, 6, 7\}\}$
		64	(65, 75, 0)	12	118.61	$(y, x) \in \{(i, \sum_{j=0}^{i-1} j) \mid i \in \{1-4, 6, 7\}\}$
		128	(65, 83, 0)	14	338.78	$(y, x) \in \{(i, \sum_{j=0}^{i-1} j) \mid i \in \{1-6\}\}$
GeoAr	$x < 4, y = 8$	16	(65, 86, 0)	15	102.84	$(y, x) \in \{(9, x) \mid x \in [0, 3]\} \cup \{(10, x) \mid x \in [9, 12]\} \cup \{(11, x) \mid x \in [19, 22]\} \cup \{(12, x) \mid x \in [30, 33]\} \cup \{(13, x) \mid x \in [44, 45]\}$
		32	(65, 127, 0)	16	147.09	$(y, x) \in \{(9, x) \mid x \in [0, 3]\} \cup \{(10, x) \mid x \in [9, 12]\} \cup \{(11, x) \mid x \in [19, 22]\} \cup \{(12, x) \mid x \in [30, 33]\} \cup \{(13, x) \mid x \in [42, 43]\} \cup \{(16, x) \mid x \in [84, 87]\} \cup \{(17, x) \mid x \in [100, 103]\} \cup \{(18, x) \mid x \in [117, 119]\}$
		64	(65, 412, 0)	33	560.45	$(y, x) \in \{(9, x) \mid x \in [0, 3]\} \cup \{(10, x) \mid x \in [9, 12]\} \cup \{(11, x) \mid x \in [19, 22]\} \cup \{(12, x) \mid x \in [30, 33]\} \cup \{(13, x) \mid x \in [42, 47]\} \cup \{(16, x) \mid x \in [84, 87]\} \cup \{(17, x) \mid x \in [100, 103]\} \cup \{(18, x) \mid x \in [117, 120]\} \cup \{(19, x) \mid x \in [135, 138]\} \cup \{(20, x) \mid x \in [154, 157]\} \cup \{(21, x) \mid x \in [175, 177]\} \cup \{(22, x) \mid x \in [198]\} \cup \{(23, x) \mid x \in [218]\}$
		128	(65, 86, 0)	14	520.94	$(y, x) \in \{(9, x) \mid x \in [0, 3]\} \cup \{(10, x) \mid x \in [9, 12]\} \cup \{(11, x) \mid x \in [19, 22]\} \cup \{(12, x) \mid x \in [30, 33]\} \cup \{(13, x) \mid x \in [42, 43]\}$
Bin0	$x = 0, y = 2, n = k$	16	(92, 52, 0)	13	85.38	$(y = 2, x = 2t), t \in [0, 16)$
		32	(91, 50, 0)	13	97.62	$(y = 2, x = 2t), t \in [0, 32)$
		64	(90, 52, 0)	14	141.35	$(y = 2, x = 2t), t \in [0, 64)$
		128	(89, 50, 0)	14	214.73	$(y = 2, x = 2t), t \in [0, 128)$
Bin0	$x < 64, y = 2, n = k$	16	(91, 53, 0)	14	91.92	$y = 2, x \in [0, 96)$
		32	(89, 50, 0)	14	110.08	$y = 2, x \in [0, 128)$
		64	(90, 51, 0)	14	149.12	$y = 2, x \in [0, 192)$
		128	(88, 48, 0)	13	194.05	$y = 2, x \in [0, 256)$
Bin1	$x = 0, n = \frac{M-k}{2}$	16	(64, 53, 0)	11	147.78	$x = 8$
		32	(64, 54, 0)	11	182.30	$x = 16$
		64	(64, 53, 0)	10	173.97	$x = 32$
		128	(64, 54, 0)	10	250.87	$x = 64$
Bin1	$x < 32, n = \frac{M-k}{2}$	16	(61, 52, 0)	14	209.65	$x \in [8, 40)$
		32	(64, 82, 0)	15	179.68	$x \in [16, 48)$
		64	(59, 48, 0)	14	173.59	$x \in [32, 64)$
		128	(59, 48, 0)	14	351.02	$x \in [64, 96)$
Bin2	$x = 0, y = 8, n < \frac{k}{2}$	16	(93, 57, 0)	17	113.90	$y = 8, x \in [8, 112)$
		32	(89, 52, 0)	14	114.14	$y = 8, x \in [0, 384)$
		64	(86, 45, 0)	13	212.68	$y = 8, x \in [0, 1024)$
		128	(84, 47, 0)	13	749.24	$y = 8, x \in [0, 4096)$
Bin2	$x < 32, y = 8, n < \frac{k}{2}$	16	(93, 55, 0)	18	120.71	$y = 8, x \in [8, 128)$
		32	(88, 47, 0)	14	115.97	$y = 8, x \in [0, 384)$
		64	(86, 50, 0)	14	183.05	$y = 8, x \in [0, 1024)$
		128	(85, 94, 0)	15	641.17	$y = 8, x \in [0, 4096)$
BiasDir	$x = 0, y = 0$	64	(2, 0, 0)	1	1.55	$(x, y) \in \{(0, 1), (1, 0)\}$
BiasDir	$x = 1, y = 1$	64	(2, 0, 0)	1	1.56	
DepRV	$x = 0, y = 0, n = k$	16	(96, 97, 0)	15	130.66	$(x, y) \in \{(i, 16 - i); i \in [5, 15]\}$
		32	(96, 135, 0)	25	265.36	$(x, y) \in \{(i, 32 - i); i \in [9, 27] \cup \{7\}\}$
		64	(96, 209, 0)	27	453.62	$(x, y) \in \{(i, 64 - i); i \in [17, 47] \cup \{14\}\}$
		128	(96, 235, 0)	43	1885.01	$(x, y) \in \{(i, 128 - i); i \in [43, 79] \cup [81, 84] \cup \{36, 88\}\}$

Prog	Pre	k	(H,#S,#P)	PH	TT	Candidate
DepRV	$x = 8, y = 16,$ $n = k$	16	(96, 105, 0)	14	130.72	$(x, y) \in \{(i, 40 - i); i \in [13, 23]\}$
		32	(96, 138, 0)	22	252.74	$(x, y) \in \{(i, 56 - i); i \in [14, 31] \cup \{11\}\}$
		64	(96, 124, 0)	18	370.04	$(x, y) \in \{(i, 88 - i); i \in [32, 47]\}$
		128	(96, 234, 0)	16	961.16	$(x, y) \in \{(i, 152 - i); i \in [57, 94]\}$
RevBin	$x < \frac{k}{2}, z = 0$	16	(64, 26, 0)	10	57.96	$z \in [1, 16)$
		32	(59, 23, 0)	7	58.81	$z \in [0, 32)$
		64	(58, 22, 0)	7	93.93	$z \in [0, 64)$
		128	(57, 22, 0)	7	187.91	$z \in [0, 128)$
RevBin	$x = \frac{k}{2}, z < 16$	16	(61, 23, 0)	8	48.02	$z \in [8, 32)$
		32	(60, 25, 0)	7	61.34	$z \in [16, 48)$
		64	(60, 23, 0)	7	101.15	$z \in [48, 80)$
		128	(60, 22, 0)	6	255.37	$z \in [64, 144)$
Mart	$c = 0, b = 64,$ $r = 0$	16	(96, 65, 0)	16	113.35	$c = 64, r \in [1, 8)$
		32	(96, 57, 0)	16	136.59	$c = 64, r \in [1, 16)$
		64	(93, 57, 0)	14	206.81	$c = 64, r \in [0, 24)$
		128	(96, 56, 0)	16	977.92	$c = 64, r \in [1, 16)$
Mart	$(c < 32, b = 64,$ $r = 32)$	16	(90, 51, 0)	14	97.37	$c \in [64, 96), r \in [33, 40) \cup [41, 48)$
		32	(90, 52, 0)	15	145.14	$c \in [64, 96), r \in [33, 40) \cup [41, 48)$
		64	(91, 50, 0)	14	319.67	$c \in [64, 96), r \in [33, 48)$
		128	(91, 52, 0)	14	1673.92	$c \in [64, 96), r \in [33, 48)$

11 Benchmark Programs

```
def Geo0(int z, bool flip):
    while (flip == 0):
        d = bernoulli(0.5)
        if (d):
            flip = 1
        else:
            z = z + 1
    return {z, flip}
```

Figure 11: Adapted from Listing 1 from Prinsys paper

```
def Geo1(int z, int x, bool flip):
    while (flip == 0):
        d = bernoulli(0.5)
        if (d):
            flip = 1
        else:
            x = x * 2
            z = z + 1
    return {z, x, flip}
```

Figure 12: Adapted from Figure 11

```
def Geo2(int z, int x, bool flip):
    while (flip == 0):
        d = bernoulli(0.5)
        if (d):
            flip = 1
        else:
            x = x + 1
            z = z + 1
    return {z, x, flip}
```

Figure 13: Adapted from Figure 11

```
def Fair(int count, bool c1, bool c2):
    while not (c1 or c2):
        c1 = bernoulli(0.5)
        if (c1):
            count = count + 1
        c2 = bernoulli(0.5)
        if (c2):
            count = count + 1
    return {count, c1, c2}
```

Figure 14: Two coins in one loop

```
def Mart(int c, int b, int rounds):
    while (b > 0):
        d = bernoulli(0.5)
        if (d):
            c = c + b
            b = 0
        else:
            c = c - b
            b = 2 * b
        rounds = rounds + 1
    return {c, b, rounds}
```

Figure 15: Martingale, Listing 4 from Prinsys paper

```
def GeoAr(int x, int y, int z):
    while (z != 0):
        y = y + 1
        d = bernoulli(0.5)
        if (d):
            z = 0
        else:
            x = x + y
    return {x, y, z}
```

Figure 16: Geometric distribution mixed with arithmetic progression

```
def Bin0(int x, int y, int n):
    while (n > 0):
        d = bernoulli(0.5)
        if (d):
            x = x + y
        n = n - 1
    return {x, n}
```

Figure 17: A plain binomial distribution

```
def Bin1(int x, int n, int M):
    while (n - M < 0):
        d = bernoulli(0.5)
        if (d):
            x = x + 1
        n = n + 1
    return {x, n}
```

Figure 18: A binomial distribution with an unchanged variable

```
def Bin2(int x, int y, int n):
    while (n > 0):
        d = bernoulli(0.5)
        if (d):
            x = x + n
        else:
            x = x + y
        n = n - 1
    return {x, n}
```

Figure 19: Binomial distribution mixed with arithmetic progression sums

```
def LinExp(int n, int count):
    while (n > 0):
        x1 = bernoulli(0.5)
        x2 = bernoulli(0.5)
        x3 = bernoulli(0.5)
        n = n - 1
        c1 = x1 or x2 or x3
        c2 = (not x1) or x2 or x3
        c3 = x1 or (not x2) or x3
        count = count + c1 + c2 + c3
    return {count, n}
```

Figure 20: Mix of binomial distribution and linearity of expectation

```
def DepRV(int x, int y, int n):
    while (n > 0):
        d = bernoulli(0.5)
        if (d):
            x = x + 1
        else:
            y = y + 1
        n = n - 1
    return {x, y, n}
```

Figure 21: Product of dependent random variables

```
def Prinsys(int x):
    while (x == 0):
        d1 = bernoulli(0.5)
        if (d1):
            x = 0
        else:
            d2 = bernoulli(0.5)
            if (d2):
                x = -1
            else:
                x = 1
    return {x}
```

Figure 22: Listing 2 from Prinsys paper

```
def Duel(bool c, bool t):
    while (c == 1):
        if (t):
            d1 = bernoulli(0.5)
            if (d1):
                c = 0
            else:
                t = not t
        else:
            d2 = bernoulli(0.5)
            if (d2):
                c = 0
            else:
                t = not t
    return {c, t}
```

Figure 23: Duelling cowboys

```
def Detm(int x, int count):
    while (x <= 10):
        x = x + 1
        count = count + 1
    return {x, count}
```

Figure 24: Deterministic loop

```
def RevBin(int x, int z):
    while (x - 1 >= 0):
        d = bernoulli(0.5)
        if (d):
            x = x - 1
            z = z + 1
    return {z, x}
```

Figure 25: A “reversed” binomial distribution

```
def BiasDir(int x, int y):
    while (x - y == 0):
        d1 = bernoulli(0.5)
        if (d1):
            x = 1
        else:
            x = 0
        d2 = bernoulli(0.5)
        if (d2):
            y = 1
        else:
            y = 0
    return {x, y}
```

Figure 26: Producing fair bits from biased coins


```

def Gambler(int x, int y, int z,
int rounds,
):
    while (0 < x and x < y):
        d = bernoulli(0.5)
        if (d):
            x = x + 1
        else:
            x = x - 1
            z = z + 1
        rounds = rounds + 1
    return {x, z, rounds}

```

Figure 27: Gambler’s ruin

References

- [Aiken *et al.*, 2016] Andrew Aiken, Marco Corradini, and Daniel Strubell. From invariant checking to invariant inference using randomized search. In *CAV 2016*, pages 388–402, 2016.
- [Bao *et al.*, 2024] Jialu Bao, Nitesh Trivedi, Drashti Pathak, Justin Hsu, and Subhajit Roy. Data-driven invariant learning for probabilistic programs. *Formal Methods in System Design*, 2024.
- [Bartocci *et al.*, 2020] Ezio Bartocci, Laura Kovács, and Miroslav Stankovič. Mora: Automatic generation of moment-based invariants. In *TACAS 2020*, pages 492–498, 2020.
- [Batz *et al.*, 2023] Kevin Batz, Mingshuai Chen, Sebastian Junges, Benjamin Lucien Kaminski, Joost-Pieter Katoen, and Christoph Matheja. Probabilistic program verification via inductive synthesis of inductive invariants. In *TACAS 2023*, pages 410–429, 2023.
- [Biere *et al.*, 1999] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. Symbolic model checking without BDDs. In *TACAS 1999*, volume 1579 of *Lecture Notes in Computer Science*, pages 193–207, 1999.
- [Biere *et al.*, 2024] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In *CAV 2024*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152, 2024.
- [Chakraborty *et al.*, 2023] Saikat Chakraborty, Shuvendu Lahiri, Sarah Fakhoury, Akash Lal, Madanlal Musuvathi, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of EMNLP 2023*, pages 9164–9175, 2023.
- [Chen *et al.*, 2015] Yu-Fang Chen, Chih-Duo Hong, Bow-Yaw Wang, and Lijun Zhang. Counterexample-guided polynomial loop invariant generation by lagrange interpolation. In *CAV 2015*, pages 658–674, 2015.
- [Das and Lal, 2017] Ankush Das and Akash Lal. Precise null pointer analysis through global value numbering. In *ATVA 2017*, pages 25–41, 2017.
- [Das *et al.*, 2015] Ankush Das, Shuvendu K. Lahiri, Akash Lal, and Yi Li. Angelic verification: Precise verification modulo unknowns. In *CAV 2015*, pages 324–342, 2015.

- [D’Souza *et al.*, 2018] Deepak D’Souza, Daniel Neider, P. Ezudheen, and Pranav Garg. Horn-ICE learning for synthesizing invariants and contracts. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):131:1–131:25, 2018.
- [Feng *et al.*, 2017] Yijun Feng, Lijun Zhang, David N. Jansen, Naijun Zhan, and Bican Xia. Finding polynomial loop invariants for probabilistic programs. In *ATVA 2017*, pages 400–416, 2017.
- [Garg *et al.*, 2014] Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *CAV 2014*, volume 8663 of *Lecture Notes in Computer Science*, pages 69–87, 2014.
- [Garg *et al.*, 2016] Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. In *POPL 2016*, pages 499–512, 2016.
- [Gehr *et al.*, 2016] Timon Gehr, Sasa Misailovic, and Martin Vechev. PSI: Exact symbolic inference for probabilistic programs. In *CAV 2016*, pages 62–83, 2016.
- [Gehr *et al.*, 2020] Timon Gehr, Samuel Steffen, and Martin Vechev. λ PSI: Exact inference for higher-order probabilistic programs. In *PLDI 2020*, pages 883–897, 2020.
- [Geldenhuys *et al.*, 2012] Jaco Geldenhuys, Matthew B. Dwyer, and Willem Visser. Probabilistic symbolic execution. In *ISSTA 2012*, pages 166–176, 2012.
- [Golia *et al.*, 2021] Priyanka Golia, Mate Soos, Sourav Chakraborty, and Kuldeep S. Meel. Designing samplers is easy: The boon of testers. In *FMCAD 2021*, pages 222–230, 2021.
- [Gretz *et al.*, 2013] Friedrich Gretz, Joost-Pieter Katoen, and Annabelle McIver. Prinsys—on a quest for probabilistic loop invariants. In *QEST 2013*, pages 193–208, 2013.
- [Joshi *et al.*, 2012] Saurabh Joshi, Shuvendu K. Lahiri, and Akash Lal. Underspecified harnesses and interleaved bugs. *ACM SIGPLAN Notices*, 47(1):19–30, 2012.
- [Kaminski and Katoen, 2017] Benjamin Lucien Kaminski and Joost-Pieter Katoen. A weakest pre-expectation semantics for mixed-sign expectations. In *LICS 2017*, pages 1–12, 2017.
- [Kozen, 1977] Dexter Kozen. Lower bounds for natural proof systems. In *FOCS 1977*, pages 254–266, 1977.
- [Kwiatkowska *et al.*, 2011] Marta Kwiatkowska, Gethin Norman, and David Parker. PRISM 4.0: Verification of probabilistic real-time systems. In *CAV 2011*, volume 6806 of *Lecture Notes in Computer Science*, pages 585–591, 2011.
- [Lahiri and Roy, 2022] Sumit Lahiri and Subhajit Roy. Almost correct invariants: Synthesizing inductive invariants by fuzzing proofs. In *ISSTA 2022*, pages 352–364, 2022.
- [Moura and Bjørner, 2008] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *TACAS 2008*, pages 337–340, 2008.
- [Naik, 2018] M. Naik. Learning loop invariants for program verification. In *NeurIPS 2018*, 2018.
- [Riley and Fedyukovich, 2022] D. Riley and G. Fedyukovich. Multi-phase invariant synthesis. In *ESEC/FSE 2022*, 2022.

- [Ruan *et al.*, 2019] Suman Ruan, Ronghui Ruan, and Ronghui Gu. CLN2INV: Learning loop invariants with continuous logic networks. *arXiv preprint arXiv:1909.11542*, 2019.
- [Savitch, 1970] Walter J. Savitch. Relationships between nondeterministic and deterministic tape complexities. *Journal of Computer and System Sciences*, 4(2):177–192, 1970.
- [Sharma *et al.*, 2013] Rahul Sharma, Saurabh Gupta, Bharath Hariharan, Alex Aiken, and Aditya V. Nori. Verification as learning geometric concepts. In *SAS 2013*, volume 7935 of *Lecture Notes in Computer Science*, pages 388–411, 2013.
- [Soos *et al.*, 2023] Mate Soos, Divesh Aggarwal, Sourav Chakraborty, Kuldeep S. Meel, and Maciej Obremski. Engineering an efficient approximate DNF-counter. In *IJCAI 2023*, pages 2031–2038, 2023.
- [Susag *et al.*, 2022] Zachary Susag, Sumit Lahiri, Justin Hsu, and Subhajit Roy. Symbolic execution for randomized programs. *Proceedings of the ACM on Programming Languages*, 6(OOPSLA2):1583–1612, 2022.
- [Wang *et al.*, 2018] Di Wang, Jan Hoffmann, and Thomas Reps. PMAF: An algebraic framework for static analysis of probabilistic programs. *ACM SIGPLAN Notices*, 53(4):513–528, 2018.
- [Wu and others, 2024] Guangyuan Wu et al. LLM meets bounded model checking: Neuro-symbolic loop invariant inference. In *ASE 2024*, pages 406–417, 2024.
- [Yao *et al.*, 2020] Suman Yao, William Ryan, and Sriram Gupta. Learning nonlinear loop invariants with gated continuous logic networks. *ACM Transactions on Programming Languages and Systems*, 42(3):18:1–18:36, 2020.
- [Yu *et al.*, 2023] Shiwen Yu, Ting Wang, and Ji Wang. Loop invariant inference through SMT solving enhanced reinforcement learning. In *ISSTA 2023*, 2023.